

Zona Docker

Volcado a PDF

Table of contents

1. Zona Docker	4
1.1 1. Intro	4
1.2 2. Amplia	4
2. Tutorial Docker	5
2.1 Tutorial Docker	5
2.2 Docker: tutorial 101	6
2.3 Ampliación tutorial introducción	18
3. Servidores	26
3.1 Servidores Web	26
3.2 Bases de Datos	28
3.3 Servidor de Aplicaciones	30
3.4 Ampliación	32
4. Ejercicios	35
4.1 Servidores	35
4.2 Crear su propia imagen	35
4.3 Configuraciones multicontenedor	35
4.4 Traefik	35
5. Traefik	36
5.1 1. Introducción	36
5.2 2. Iniciar Traefik	37
5.3 3. Descubriendo servicios	38
5.4 Ampliando el ejemplo inicial	38
5.5 4. Escalar el servicio: balanceo de carga	41
5.6 5. Ejercicio propuesto	41
5.7 6. Enlaces	41
5.8 7. Anexo: Fichero <code>docker-compose.yml</code> completo	41
6. Cluster: Docker Swarm	43
6.1 1. Introducción	43
6.2 2. Preparación del <i>swarm</i>	43
6.3 3. Desplegar servicio	44
6.4 4. Herramienta gráfica: portainer	46
6.5 5. Anexos	47
7. Fuentes ejemplos o ejercicios	49
7.1 1. Código ejemplos tutorial	49
7.2 2. Código ejemplos de sitios web	49

7.3	3. Código ejemplo tomcat	49
7.4	4. Scripts SQL	49
7.5	5. Docker Swarm	49
8.	Seguridad en Docker	50
8.1	1. Seguridad en el host	50
8.2	2. Seguridad en el demonio docker	50
8.3	3. Seguridad en la ejecución de contenedores	50
8.4	4. Seguridad en la construcción de imágenes	52
8.5	5. Mejorando el fichero dockerfile	53
8.6	6. Escaneo de seguridad de las imágenes	56
8.7	7. Hardened images	58
8.8	8. Varios de ampliación	59
9.	Amplia Compose	61
9.1	Encadenar arranque contenedores	61
9.2	Complementos y ayudas	63

1. Zona Docker

1.1 1. Intro

Los contenidos de esta zona o bloque son:

- Se inicia con tutorial docker de la propia empresa [docker.com](https://docs.docker.com/)
- Se refuerza con ejercicios de despliegue de servidores, CMS, etc.
- Se introduce el uso de `traefik` como proxy inverso en un sistema en local de contenedores,
- Se muestra un cluster `docker swarm`.

1.2 2. Amplia

- Desplegar un entorno de contenedores docker con `traefik` en una instancia AWS con certificados SSL. Para ello:
 - a. [Uso de AWS](#)
 - b. [Traefik y https](#)

2. Tutorial Docker

2.1 Tutorial Docker

2.1.1 1. Instalar docker engine

1. Instalar docker engine en la máquina virtual: [Install Docker Engine on Debian](#)
2. Realizar la post-instalación: [Manage Docker as a non-root user](#)

2.1.2 2. Realizar paso a paso el tutorial docker 101

If you wish to run the tutorial, you can use the following command after installing

```
docker run -d -p 80:80 docker/getting-started
```

Once it has started, you can open your browser to <http://localhost>.

Además dispone en la plataforma de un curso de *Introducción a Docker*.

2.1.3 3. Típicos problemas en el aula

- Levantar un contenedor con un nombre que ya está siendo usado por otro contenedor (activo o no).
- Levantar un contenedor mapeando un puerto del host que ya está siendo usado por otro contenedor.
- Ojo con la "cache" del navegador, puede ser necesario forzar la recarga de la página, o usar el comando `curl`, etc.
- Existen dos versiones del tutorial docker, y el código de la aplicación de ejemplo que se utiliza es diferente en cada versión. No puede mezclar la aplicación de un tutorial con los comandos o el `docker-compose.yml` del otro. Las dos versiones del tutorial son:
- `docker/getting-started:latest` (usa node 18 y mysql 8)
- `docker/getting-started:pwd` (usa node 10 y mysql 5.7)
- Cuidado al crear y usar un volumen para una versión de una base de datos e intentar usar ese volumen con otra versión de la base de datos (o con otra base de datos).

2.2 Docker: tutorial 101

2.2.1 1. Instalación

Puede instalar Docker directamente en su equipo o en una máquina virtual con Linux. Puede acceder a <https://docs.docker.com/engine/install/> para obtener instrucciones según su distribución. En los ejemplos se supone una instalación en Debian.

En las prácticas y por comodidad se usa el *script* que proporciona Docker para una instalación de desarrollo:

```
$ curl -fsSL https://get.docker.com -o get-docker.sh
$ sudo sh get-docker.sh
Executing docker install script, commit:
7cae5f8b0decc17d657fbc13b2737
<...>
```

1.1. post-installation steps

Si quiere que el usuario ejecute los comandos `docker` sin usar `sudo`

If you would like to use Docker as a non-root user, you should now consider adding your user to the “docker” group with something like:

```
$sudo groupadd docker
$sudo usermod -aG docker <your-user>
```

Remember to log out and back in for this to take effect!. >You can also run the following command to activate the changes to groups:

```
newgrp docker
```

No olvide comprobar que la instalación es correcta lanzando la imagen `hello-world`:

```
$docker run hello-world
```

2.2.2 2. Introducción

Docker proporciona un tutorial guiado para entender como es el despliegue de una aplicación en un entorno de contenedores (en ese tutorial se utilizará una aplicación Node):

<https://www.docker.com/101-tutorial/>

El tutorial se ofrece en una imagen Docker que debe iniciar con el comando `docker run -d -p 80:80 docker/getting-started`

```
alu@xdebian11:~$ docker run -d -p 80:80 docker/getting-started
Unable to find image 'docker/getting-started:latest' locally
latest: Pulling from docker/getting-started
c158987b0551: Pull complete
...
4f7e34c2de10: Pull complete
Digest: sha256:d79336f4812b65.....71b414fbd9297609ffb989abc1
Status: Downloaded newer image for docker/getting-started:latest
670fe226d2779c2c96e6417aabf34.....6d7ca4d050e0161e296325c3b7
alu@xdebian11:~$
```

Y puede acceder al tutorial en la URL <http://localhost>

```
alu@xdebian11:~$ firefox localhost:80
alu@xdebian11:~$
```

The screenshot shows the Docker Labs 'Getting Started' tutorial page. The sidebar on the left lists various topics under 'Getting Started'. The main content area has a heading 'Getting Started' and a section 'The command you just ran' which provides the command to run the container. A 'Pro tip' box is also visible.

Getting Started

[Getting Started](#)

Our Application

Updating our App

Sharing our App

Persisting our DB

Using Bind Mounts

Multi-Container Apps

Using Docker Compose

Image Building Best Practices

What Next?

Getting Started

The command you just ran

Congratulations! You have started the container for this tutorial! Let's first explain the command that you just ran. In case you forgot, here's the command:

```
docker run -d -p 80:80 docker/getting-started
```

You'll notice a few flags being used. Here's some more info on them:

- `-d` - run the container in detached mode (in the background)
- `-p 80:80` - map port 80 of the host to port 80 in the container
- `docker/getting-started` - the image to use

Pro tip

Table of contents

- The command you just ran
- The Docker Dashboard
- What is a container?
- What is a container image?

Ya sólo queda seguir los pasos que propone el tutorial.

2.2.3 3. La aplicación

El tutorial trabaja con una aplicación en node. Los pasos a dar para construir una imagen Docker que incluya esa aplicación están en el tutorial y son:

- Descargar la aplicación (en formato ZIP) desde el navegador. Descomprimirla en el directorio de inicio de sesión. Se obtiene un directorio `app`, y dentro los directorios `spec` y `src`.
- Abrir el directorio `app` con el editor `vscode`.
- Crear dentro de `app` un fichero de nombre `Dockerfile`

```
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
```

- Acceder a un terminal en el directorio `app` y construir la imagen (nombre `getting-started`) que incluye la aplicación con `docker build`:

```
alu@xdebian11:~/app$ ls
Dockerfile package.json spec src yarn.lock
alu@xdebian11:~/app$ docker build -t getting-started .
[+] Building 15.1s (7/8)
...
```

- Como resultado final tenemos una imagen Docker que incluye la aplicación. Esta accede a una base de datos en la propia imagen usando SQLite.

```
alu@xdebian11:~/app$ docker images
REPOSITORY          TAG       IMAGE ID   CREATED   SIZE
getting-started      latest   47e87..92  2 minutes ago 262MB
docker/getting-started latest   3e439..2f  6 weeks ago  47MB
```

```
hello-world          latest feb5d..a5 16 months ago 13.3kB
alu@xdebian11:~/app$
```

- Iniciar el contenedor con `docker run`

```
alu@xdebian11:~/app$ docker run -dp 3000:3000 --name miapp getting-started
f412f7de1737bce2c43490196d0....afa03d1dffaa7cec
alu@xdebian11:~/app$
```

- Y probar la aplicación que es accesible en el puerto 3000: <http://localhost:3000> añadiendo datos:

```
alu@xdebian11:~/app$ firefox http://localhost:3000
```

Puede ahora parar el contenedor y ver como la aplicación deja de estar disponible:

```
alu@xdebian11:~/app$ docker stop miapp
miapp
alu@xdebian11:~/app$
```

Y volver a arrancar el contenedor y ver que la aplicación y los datos están disponibles:

```
alu@xdebian11:~/app$ docker start miapp
miapp
alu@xdebian11:~/app$
```

2.2.4 4. Actualizar la app

Se propone ahora modificar la aplicación. Tras modificarla habrá que construir una nueva imagen, lanzarla, probarla, etc. Pasos a dar:

- Hacer un cambio en la aplicación: modificar en el fichero `app/src/static/js/app.js` el mensaje que se le indica en la línea 56.
- Construir una nueva imagen con la aplicación actualizada (si no se usan etiquetas se pierde el tag (`latest`) de la imagen inicial y pasa a ser "`none`"):
 - Si intenta lanzar la nueva imagen es posible que tenga un error ya que si no hemos parado el contenedor previo el nuevo intenta usar el mismo puerto

```
alu@xdebian11:~/app$ docker build -t getting-started .
[+] Building 11.3s (9/9) FINISHED
...
=> => writing image sha256:d85186b3b7dc7a088d49318bc4f2a177a4bcd2f677a34271 0.0s
=> => naming to docker.io/library/getting-started 0.0s
alu@xdebian11:~/app$ docker images
REPOSITORY          TAG       IMAGE ID   CREATED      SIZE
getting-started      latest    d85...dc  10 seconds ago 262MB
<none>              <none>    47e...92  55 minutes ago 262MB
docker/getting-started latest    3e4...2f  6 weeks ago  47MB
hello-world         latest    feb...a5  16 months ago 13.3kB
alu@xdebian11:~/app$
```

- Si intenta lanzar la nueva imagen es posible que tenga un error ya que si no hemos parado el contenedor previo el nuevo intenta usar el mismo puerto

```
alu@xdebian11:~/app$ docker run -d -p 3000:3000 --name nuevaapp getting-started
f94faea80f45306db4f387.....6ffba3753541419cebc316a3179cb26
docker: Error response from daemon: driver failed programming external connectivity on endpoint nuevaapp
(312d6bfa399ecc15947d3.....c21506867769b01cdc2c2ebc68071c):
Bind for 0.0.0.0:3000 failed: port is already allocated.
alu@xdebian11:~/app$
```

- Puede comprobar los contenedores en ejecución con `docker ps`, detenerlo con `docker stop` y borrarlo con `docker rm` para terminar lanzando el nuevo contenedor:

```
alu@xdebian11:~/app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
c75ba15aa019   47e872a9bc92   "docker-entrypoint.s..." 15 minutes ago Up 11 minutes 0.0.0.0:3000->3000/tcp, :::3000->3000/tcp  miapp
292ffb5d2de8   docker/getting-started "/docker-entrypoint..." 24 minutes ago Up 24 minutes 0.0.0.0:80->80/tcp, :::80->80/tcp      tutorial
alu@xdebian11:~/app$ docker stop miapp
miapp
alu@xdebian11:~/app$ docker rm miapp
miapp
alu@xdebian11:~/app$ docker run -d -p 3000:3000 --name nuevaapp getting-started
f48e6b71bdafb4fd84c02e4a9ab.....74c518545599847cb66cc7cbd
alu@xdebian11:~/app$
```

Nota: puede generar una imagen especificando una etiqueta completa con el formato `<repo:tag>`: `docker build -t getting-started:v1.0.`

2.2.5 5. Compartir en Dockerhub

En este apartado "registrará" su imagen en DockerHub. Para ello es necesario tener una cuenta en DockerHub. La creación previa del repositorio es opcional, si no lo hace se creará al subir la imagen un repositorio público (y sin descripción).

Pasos a dar:

- Crear cuenta en DockerHub si no la tuviera.
- Iniciar sesión en DockerHub desde línea de comandos con `docker login -u <dockerhub_username>`
- Etiquetar correctamente la imagen con su usuario en Dockerhub `docker tag repo user/repo`.

```
docker tag getting-started user/getting-started
```

- Subir la imagen a DockerHub con `docker push`:

```
docker push user/getting-started
```

Para poder utilizar dicha imagen basta indicar el repositorio de la imagen al ejecutarla (incluyendo el usuario). Si no está en local se descarga de DockerHub.

```
docker run -dp 3000:3000 usuario/getting-started
```

2.2.6 6. Persistencia de los datos

Ahora mismo los datos del contenedor se pierden al terminar el mismo:

When a container runs, it uses the various layers from an image for its filesystem. Each container also gets its own "scratch space" to create/update/remove files. Any changes won't be seen in another container, even if they are using the same image.

En el caso de nuestra aplicación al terminar la ejecución del contenedor se pierde el fichero con la base de datos SQLite

the todo app stores its data in a SQLite Database at /etc/todos/todo.db. If you're not familiar with SQLite, no worries! It's simply a relational database in which all of the data is stored in a single file.

Con los volúmenes va a conectar el sistema de ficheros del contenedor con el sistema de ficheros de la maquina host. De esta forma, si monta el mismo directorio del host al relanzar el contenedor accederá siempre a los mismos ficheros. Hay dos formas de usar volúmenes:

- volúmenes con nombre (gestionados por Docker)
- *bind mounts* o volúmenes en directorios

6.1. Volumen con nombre

Al crear volúmenes con nombre Docker se encarga de reservar el espacio necesario en el sistema de ficheros del host y hacer el volumen accesible al contenedor.

Pasos a dar:

1. Crear un volumen con el nombre `todo-db`

```
docker volume create todo-db
```

2. Lanzar el contenedor con un nuevo parámetro, la opción `-v`. A esta se le indican dos valores, el volumen con nombre creado `todo-db` y el directorio `/etc/todos` del contenedor donde se montará el volumen (la aplicación del ejemplo almacena en `/etc/todos` la base de datos)

```
docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
```

3. Acceda a la aplicación desde el navegador y añada varios elementos.
4. Detenga y borre el contenedor:

```
docker rm -f getting-started
```

5. Inicie de nuevo otro contenedor y compruebe que se conservan los datos de la ejecución anterior.

```
docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
```

6. Para ver la localización del volumen use el comando `docker volume inspect`

```
docker volume inspect todo-db
```

7. Por último, puede borrar el volumen con `docker volume rm` (genera error si el volumen está en uso)

console

```
alu@xdebian11:~/app$ docker volume rm todo-db
Error response from daemon: remove todo-db: volume is in use - [c89f805e963720.....d1c00ba2a88696fc59f9b8c525e]
alu@xdebian11:~/app$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
c89f805e9637	getting-started	"docker-entrypoint.s..."	About a minute ago	Up About a minute 0.0.0.0:3000->3000/tcp, :::3000->3000/tcp
292ffb5d2de8	docker/getting-started	"/docker-entrypoint...."	About an hour ago	Up About an hour 0.0.0.0:80->80/tcp, :::80->80/tcp

```
alu@xdebian11:~/app$ docker rm -f ecstatic_johnson
ecstatic_johnson
alu@xdebian11:~/app$ docker volume rm todo-db
todo-db
alu@xdebian11:~/app$
```

6.2. Bind mount

Los volúmenes con nombre son suficientes si queremos almacenar datos y no le preocupa dónde ni cómo se almacenan. Pero puede ser que busque un control mayor de esos datos, e interese que un directorio determinado del host se monte como un directorio del contenedor. Esto es útil en entornos de desarrollo.

La idea de este apartado es hacer cambios en la aplicación (directorio `app`) y que estos se reflejen en el contenedor en ejecución sin tener que crear una nueva imagen, ejecutarla, etc.

Nota: en este apartado la aplicación Node usa el módulo `nodemon`. Este detecta cambios en los ficheros de la aplicación Node y si es necesario la reinicia. Puede comprobar su uso en el fichero `package.json`.

Para probar estos volúmenes se lanza el contenedor mapeando (opción `-v`) el directorio de trabajo (debe ser `app`) al directorio `/app` del contenedor. En este caso no se usa la imagen creada en apartados previos sino una imagen con Node (`node:18-alpine`)

```
docker run -dp 3000:3000 \
  -w /app \
  -v "${PWD}:/app" \
  node:18-alpine \
  sh -c "yarn install && yarn run dev"
```

La opción `-w` nos indica el directorio de trabajo en el contenedor (similar a `WORKDIR /app` en el fichero `Dockerfile`). Si ahora modifica por ejemplo el texto del botón *"Add item"* en el fichero `src/static/js/app.js` puede observar como se "refresca" la aplicación con ese cambio.

Incluso puede combinar los dos tipos de volúmenes:

- un *bind mount* para la aplicación
- un volumen con nombre para dar persistencia a la BD:

```
docker run -dp 3000:3000 \
  -w /app \
  -v "$(pwd):/app" \
  -v todo-db:/etc/todos \
  node:18-alpine \
  sh -c "yarn install && yarn run dev"
```

Puede ver el fichero de log con `docker logs -f <container_id>`

```
alu@xdebian11:~/app$ docker logs -f nervous_heisenberg
yarn install v1.22.19
[1/4] Resolving packages...
...
[4/4] Building fresh packages...
Done in 11.71s.
yarn run v1.22.19
$ nodemon src/index.js
[nodemon] 2.0.20
...
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node src/index.js`
Using sqlite database at /etc/todos/todo.db
Listening on port 3000
[nodemon] restarting due to changes...
[nodemon] starting `node src/index.js`
Using sqlite database at /etc/todos/todo.db
Listening on port 3000
```

2.2.7 7. Multicontenedor modo "manual"

En este apartado se introducen varios cambios:

1. Pasar la aplicación de un único contenedor a dos:
2. un contenedor con la base de datos, que pasa de SQLite a MySQL.
3. otro contenedor con la aplicación
4. Usar la herramienta `docker compose` para describir nuestra aplicación.

La aplicación es capaz de diferenciar el paso de uno a dos contenedores en base a la existencia de la variable de entorno `MYSQL_HOST=mysql` que comprueba en el fichero `src/persistence/index.js`

```
// file src/persistence/index.js
if (process.env.MYSQL_HOST)
  module.exports = require('./mysql');
else
  module.exports = require('./sqlite');
```

Realice el cambio a dos contenedores paso a paso ejecutando cada uno de ellos de forma manual.

1. Crear una red (`docker network create`) donde se comunicará los dos contenedores:

```
alu@xdebian11:~/app$ docker network create todo-app
b5d1c5cccc26f02861411b845...0fb64ccbc3067a99d9e6cb9cadfc17
alu@xdebian11:~/app$
```

2. Arrancar un contenedor con un servidor MySQL. Al mismo se le pasan dos variables de entorno, la clave de MySQL y el nombre de la base de datos. Tiene los detalles de cómo lanzar una imagen MySQL en este enlace: (https://hub.docker.com/_/mysql/). Observe que también se usa un volumen `todo-mysql-data` para darle persistencia a la base de datos:

```
alu@xdebian11:~/app$ docker run -d \
  --network todo-app --network-alias mysql \
  -v todo-mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=todos \
  --name mysqlcontainer
```

```
mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
39fbafb6c7ef: Pull complete
...
Digest: sha256:03b0af...9e0dcaebe411a1ec75e6adc9fea051aa2
Status: Downloaded newer image for mysql:8.0
```

```
04e7b5d6bc89883ba9f1c...79367b4cfaa19ba1bf2e68160b389b2346  
alu@xdebian11:~/app$
```

1. Probar la creación de la base de dato accediendo al contenedor con `docker exec -it` y ejecutamos el comando `mysql -p`

```
alu@xdebian11:~/app$ docker exec -it mysqlserver mysql -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.32 MySQL Community Server - GPL

...

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| todos |
+-----+
5 rows in set (0.00 sec)

mysql> use todos;
Database changed
mysql> show tables;
Empty set (0.00 sec)

mysql> exit
Bye
alu@xdebian11:~/app$
```

2. En el tutorial se muestra el uso de la imagen `nicolaka/netshoot` para obtener la IP del servidor MySQL y comprobar que la opción `--network-alias mysql` permite usar `mysql` como nombre de host del contenedor.

```
alu@xdebian11:~/app$ docker inspect mysqlserver | grep IPAddress
    "SecondaryIPAddresses": null,
    "IPAddress": "",
    "IPAddress": "172.19.0.3",
alu@xdebian11:~/app$ docker run -it --network todo-app nicolaka/netshoot
      dP      dP      dP
      88      88      88
88d888b. .d8888b. d8888P .d8888b. 88d888b. .d8888b. d8888P
88'  '88 88oooo8 88  Y8oooo. 88'  '88 88'  '88 88'  '88 88
88  88 88.   . 88  88 88 88  88 88.   .88 88.   .88 88
dP   dP '88888P' dP   '88888P' dP   dP '88888P' '88888P' dP

Welcome to Netshoot! (github.com/nicolaka/netshoot)
5a5c709c3917 ㉿ ~ ㉿ dig mysql

; <<>> DiG 9.18.8 <<>> mysql
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 51505
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;mysql.      IN A

;; ANSWER SECTION:
mysql.      600 IN A 172.19.0.3

;; Query time: 0 msec
;; SERVER: 127.0.0.11#53(127.0.0.11) (UDP)
;; WHEN: Sun Feb 05 14:06:32 UTC 2023
;; MSG SIZE rcvd: 44

5a5c709c3917 ㉿ ~ ㉿ ping mysql
PING mysql (172.19.0.3) 56(84) bytes of data.
64 bytes from mysqlserver.todo-app (172.19.0.3): icmp_seq=1 ttl=64 time=0.131 ms
64 bytes from mysqlserver.todo-app (172.19.0.3): icmp_seq=2 ttl=64 time=0.096 ms
^C
--- mysql ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1013ms
rtt min/avg/max/mdev = 0.096/0.113/0.131/0.017 ms

5a5c709c3917 ㉿ ~ ㉿ exit
alu@xdebian11:~/app$
```

3. Lanzar un segundo contenedor con la aplicación y especificar las variables de entorno que permiten conectarla con la base de datos `mysql`:

```
alu@xdebian11:~/app$ docker run -dp 3000:3000 \
  -w /app -v "$(pwd):/app" \
  --network todo-app \
  -e MYSQL_HOST=mysql \
  -e MYSQL_USER=root \
  -e MYSQL_PASSWORD=secret \
  -e MYSQL_DB=todos \
  --name miapp \
```

```
node:18-alpine \
sh -c "yarn install && yarn run dev"
5ab724f02d8b3aba8d40e....24bc8581a8a1730fb8d089e0441
alu@xdebian11:~/app$
```

4. Comprobar que se ha conectado con `mysql`

```
alu@xdebian11:~/app$ docker logs miapp
yarn install v1.22.19
[1/4] Resolving packages...
success Already up-to-date.
Done in 0.38s.
yarn run v1.22.19
$ nodemon src/index.js
[nodemon] 2.0.20
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node src/index.js`
Waiting for mysql:3306.
Connected!
Connected to mysql db at host mysql
Listening on port 3000
alu@xdebian11:~/app$
```

5. Probar la aplicación añadiendo datos.

6. Conectar a la base de datos para comprobar los datos en la tabla `todo_items`:

```
alu@xdebian11:~/app$ docker exec -it mysqlserver mysql -p todos
Enter password:
....
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> select * from todo_items;
+-----+-----+-----+
| id          | name                | completed |
+-----+-----+-----+
| e1b61a....891d | Revisar Apuntes Docker | 0        |
| 867350....6769 | Rehacer deploy Heroku  | 0        |
| ee1523....d230 | Probar deploy en AWS   | 0        |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> exit
Bye
alu@xdebian11:~/app$
```

Nota: El uso de variables de entorno para información sensible no es adecuado. Verá como resolver esta situación en un apartado posterior.

2.2.8 8. Multi contenedor con docker-compose

En vez de realizar el despliegue de forma manual paso a paso lo habitual es utilizar un fichero descriptivo que es interpretado por el comando `docker compose`.

8.1. Fichero `docker-compose.yml`

El fichero sería similar a este (en el tutorial se explica paso a paso su creación):

```
services:
  app:
    image: node:18-alpine
    command: sh -c "yarn install && yarn run dev"
    ports:
      - 3000:3000
    working_dir: /app
    volumes:
      - ./:/app
    environment:
      MYSQL_HOST: mysql
      MYSQL_USER: root
      MYSQL_PASSWORD: secret
      MYSQL_DB: todos

  mysql:
    image: mysql:8.0
    volumes:
      - todo-mysql-data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: todos
```

```
volumes:
  todo-mysql-data:
```

8.2. Comandos docker compose

La herramienta `docker compose` debe estar instalada. Para verificarlo:

```
alu@xdebian11:~/app$ docker compose version
Docker Compose version v2.15.1
alu@xdebian11:~/app$
```

Ahora para lanzar la aplicación basta ejecutar el comando `docker-compose up -d` (crea la red, los volúmenes necesarios y lanza los contenedores), Si no se especifica fichero se utiliza el `docker-compose.yml` del directorio:

```
alu@xdebian11:~/app$ docker compose up -d
[+] Running 4/4
:: Network app_default          Created 0.1s
:: Volume "app_todo-mysql-data" Created 0.0s
:: Container app-app-1          Started 0.7s
:: Container app-mysql-1        Started 0.6s
alu@xdebian11:~/app$
```

Para ver los logs de los dos servicios en "vivo" basta ejecutar `docker-compose logs -f`. Si desea ver solo el log de uno de los servicios en este caso `docker-compose logs app|mysql`.

Para detener los servicios basta `docker-compose down`. Tenga en cuenta que por defecto este comando no borra los volúmenes (tendría que usar el flag `--volumes`).

```
alu@xdebian11:~/app$ docker compose down
[+] Running 3/3
:: Container app-mysql-1        Removed   3.4s
:: Container app-app-1          Removed   0.2s
:: Network app_default          Removed   0.1s
alu@xdebian11:~/app$ docker volume ls
DRIVER      VOLUME NAME
local       app_todo-mysql-data
alu@xdebian11:~/app$
```

2.2.9 9. Image Building Best Practices

9.1. Escanear la imagen

En el tutorial de docker habla de `docker scan`, que ha sido sustituida por `docker scout`. Para instalarlo: <https://docs.docker.com/scout/install/>

Y para utilizarlo [en línea de comandos](#). Algunos ejemplos de uso:

- `docker scout quickview`: summary of the specified image, see Quickview
- `docker scout cves`: local analysis of the specified image, see CVEs
- `docker scout compare`: analyzes and compares two images

También es posible (opción de pago, free sólo una) activar el escaneo de imágenes al registrarlas en DockerHub. [Docker Scout image analysis](#)

9.2. Optimizar *build* de la imagen

Las imágenes de Docker se organizan en capas. Si un nivel o capa cambia provoca que todas las capas posteriores se tengan que reconstruir. Por ello es preferible situar primero las capas que con menos frecuencia cambian, y dejar para las últimas las que se modifican con más frecuencia.

A continuación se modifica el Dockerfile del tutorial: dejamos como penúltima capa la copia del directorio de la aplicación y se crean antes la copia de los paquetes de dependencias. Se supone que habrá mas cambios en el código que en las dependencias.

```
FROM node:18-alpine
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --production
COPY . .
CMD ["node", "src/index.js"]
```


9.3. El fichero .dockerignore

Este fichero permite indicar directorios (y ficheros) que no se deben copiar del host a la imagen durante la creación de la imagen. El formato es similar al fichero `.gitignore` utilizado con `git`. En el tutorial se crea este fichero y se incluye el directorio `node_modules` que se genera durante las pruebas en local de la aplicación.

9.4. Build multistage

Esta facilidad se usa para separar las dependencias de construcción de la aplicación de las dependencias de ejecución de la misma. Esto reduce el tamaño de la imagen ya que sólo se incluye lo necesario para su ejecución.

Ejemplo típico son las aplicaciones Java, donde JDK es necesario para generar la aplicación en bytecode, pero no para ejecutarla. O utilizar herramientas adicionales para construir la aplicación que tampoco serían necesarias para su ejecución.

Vea este fichero Dockerfile

```
FROM maven AS build
WORKDIR /app
COPY . .
RUN mvn package

FROM tomcat
COPY --from=build /app/target/file.war /usr/local/tomcat/webapps
```

La primer parte construye el fichero WAR en una imagen `maven`. La segunda parte usa una imagen `tomcat` y simplemente copia el fichero WAR generado en la primera parte en el directorio de aplicaciones de `tomcat`. Observe que el fichero se copia `--from=build`, alias que se genera en la primera línea (`FROM maven AS build`). En este caso la imagen `tomcat` puede ser un 10% menor que `maven`.

2.2.10 10. Anexos

10.1. Obsoleto: docker scan

Existen herramientas para comprobar posibles vulnerabilidades en imágenes. Una de ellas es Snyk.

Docker has partnered with Snyk to provide the vulnerability scanning service. <https://snyk.io/product/container-vulnerability-management/>

Para escanear la imagen desde CLI:

```
alu@xdebian11:~/app$ docker scan docker/getting-started

Testing docker/getting-started...

x Low severity vulnerability found in libxpm/libxpm
Description: CVE-2022-4883
Info: https://security.snyk.io/vuln/SNYK-ALPINE317-LIBXPM-3232761
Introduced through: libxpm/libxpm@3.5.14-r0, gd/libgd@2.3.3-r3
From: libxpm/libxpm@3.5.14-r0
From: gd/libgd@2.3.3-r3 > libxpm/libxpm@3.5.14-r0
Fixed in: 3.5.15-r0

....

x High severity vulnerability found in curl/libcurl
Description: Cleartext Transmission of Sensitive Information
Info: https://security.snyk.io/vuln/SNYK-ALPINE317-CURL-3179543
Introduced through: curl/libcurl@7.86.0-r1, curl/curl@7.86.0-r1
From: curl/libcurl@7.86.0-r1
From: curl/curl@7.86.0-r1 > curl/libcurl@7.86.0-r1
From: curl/curl@7.86.0-r1
Fixed in: 7.87.0-r0

Organization:    angoes-98m
Package manager: apk
Project name:    docker-image|docker/getting-started
Docker image:    docker/getting-started
Platform:        linux/amd64
Base image:      nginx:1.23.3-alpine
Licenses:        enabled

Tested 63 dependencies for known issues, found 6 issues.

Your base image is out of date
1) Pull the latest version of your base image by running 'docker pull nginx:1.23.3-alpine'
2) Rebuild your local image
```

2.3 Ampliación tutorial introducción

2.3.1 1. Docker Secret

Imagine un despliegue con una base de datos `postgres` y `adminer` (herramienta de gestión de bases de datos de código abierto, gratuita y basada en PHP). Necesita indicar el usuario, clave y base de datos al servidor `postgres`. La primera solución (no recomendable en un entorno de producción) es introducir esos valores en el fichero `compose-postgres.v0.yml`:

```
version: '3.1'
services:
  db:
    image: postgres
    container_name: postgres-container
    environment:
      POSTGRES_USER: pps
      POSTGRES_PASSWORD: ppspassword
      POSTGRES_DB: ppsDB

  adminer:
    image: adminer
    ports:
      - 8080:8080
```

Hacerlo de esta forma tiene el inconveniente de hacer visible esa información sensible. Para gestionar este tipo de información en el despliegue se puede usar `docker secret`.

<https://docs.docker.com/engine/swarm/secrets> a secret is a blob of data, such as a password, SSH private key, SSL certificate, or another piece of data that should not be transmitted over a network or stored unencrypted in a Dockerfile or in your application's source code. You can use Docker secrets to centrally manage this data and securely transmit it to only those containers that need access to it. Secrets are encrypted during transit and at rest in a Docker swarm. A given secret is only accessible to those services which have been granted explicit access to it, and only while those service tasks are running.

Se puede crear un secreto de dos formas:

1. Usando el comando `docker secret create` para crear primero el secreto y luego usarlo en el despliegue.
2. A partir de un secreto almacenado en fichero y crear de forma automática el secreto en el momento del despliegue.

Debe tener en cuenta un detalle: `docker secret` es un servicio que se debe ejecutar en un cluster, luego debe iniciar primero un cluster (en este caso de un nodo) con `docker swarm init`. El uso de `docker swarm` se incluye en otra práctica.

```
$ docker swarm init
....
$
```

1.1. Primer ejemplo: secretos generado por comando

En el despliegue de `postgres` y `adminer` primero se crean con el comando `docker secret create` tres secretos de nombres `pg_user`, `pg_password` y `pg_database`:

```
$ echo "pps" | docker secret create pg_user -
nqfkiha5j5n4z1d72via08rq
$ echo "ppspassword" | docker secret create pg_password -
p02zpc0uqjm01b1lyfb76gvdv
$ echo "ppsDB" | docker secret create pg_database -
o6o15f0e6vo4da0er7wq7j9j1
$
```

Puede listar los secretos creado con el comando `docker secret ls`:

```
$ docker secret ls
ID          NAME          DRIVER  CREATED    UPDATED
o6...j1    pg_database    28 s ago 28 s ago
p0...dv    pg_password    39 s ago 39 s ago
nq...rq    pg_user        9 s ago  49 s ago
d$
```

Una vez creados los secretos se modifica el fichero compose que los utilice. Los secretos por defecto se montan en el directorio `/run/secrets` del contenedor. Por eso las variables de entorno del servicio `db` en el siguiente fichero compose se asocian a los ficheros `/run/secrets/pg_user`, etc. En el bloque de `secrets` del fichero compose se indica su procedencia *externa*.

En este caso el fichero `compose-postgres.v1.yml` será similar a este:

```
services:
  db:
    image: postgres
    restart: always
    environment:
      POSTGRES_USER_FILE: /run/secrets/pg_user
      POSTGRES_PASSWORD_FILE: /run/secrets/pg_password
      POSTGRES_DB_FILE: /run/secrets/pg_database
    secrets:
      - pg_password
      - pg_user
      - pg_database
    ....

# tienen que estar creados previamente
secrets:
  pg_user:
    external: true
  pg_password:
    external: true
  pg_database:
    external: true
```

En un cluster la forma de desplegar es con el comando `docker stack deploy`, indicando el nombre del fichero con la opción `-c file` y el nombre del stack en este caso `demo`:

```
$ docker stack deploy -c compose_postgres.v1.yml -d demo
Creating network demo_default
Creating service demo_db
Creating service demo_adminer
$
```

Y ahora puede acceder al gestor `adminer`:

3 Login - Adminer

Login - Adminer x +

Login - Adminer 192.168.71.129:8080

Idioma: Español v

Adminer 4.8.1

Login

Motor de base de datos	PostgreSQL v
Servidor	db
Usuario	myuser
Contraseña	●●●●●●●●●●●●●●●●
Base de datos	mydatabase

☐ Guardar contraseña

Idioma: Español

Adminer 4.8.1

Login

Motor de base de datos	PostgreSQL
Servidor	db
Usuario	myuser
Contraseña	●●●●●●●●●●●●●●●●●●●●
Base de datos	mydatabase

☐ Guardar contraseña

Para terminar hay que eliminar el despliegue del stack (`docker stack rm <stack>`) y desactivar el cluster (`docker swarm leave`):

```
$ docker stack rm demo
Removing service demo_adminer
Removing service demo_db
Removing network demo_default
$ docker swarm leave --force
Node left the swarm.
$
```

1.2. Segundo ejemplo: secretos en fichero

En este ejemplo se despliega el CMS `wordpress` que utiliza una base de datos `mysql`. En este caso los secretos se utilizan en los dos servicios del despliegue, y **se van a almacenar en ficheros externos al contenedor**. A estos ficheros docker accede en el momento del despliegue para leer su contenido y crear de forma automática los secretos.

Primero se crean los ficheros con la información sensible. Se usa `openssl` para crear las claves (si desea modificar la longitud de la clave modifique el último valor de `openssl`) y se redirige la salida a un fichero. El usuario y la base de datos no se han considerado información sensible en este ejemplo.

Se crean dos ficheros: uno para la clave de root mysql y otro para la clave del usuario en mysql/wordpress.

```
$ openssl rand -base64 30 > db_root_password.txt
$ openssl rand -base64 30 > db_user_password.txt
$ cat dd_*.txt
i8Z+9B78TV4j12IeeCVfIWj7j+JLvGI1aq/fLsp
/J9+qoBVHtFvvKi85LA+EcAAVu5NovFf1b2Uyp5c
$
```

Ya se ha comentado que los secretos están disponibles para los servicios en el directorio `/run/secrets` del contenedor. Lo que cambia en este segundo ejemplo es el apartado final donde se indica para cada secreto el fichero que le corresponde. Tenga en cuenta que los valores del usuario (`pps`) y su clave (`/run/secrets/db_password`) deben coincidir en los dos servicios. El fichero `compose_wordpress.yml` queda:

```
services:
  db:
```

```

image: mysql:latest
volumes:
  - db_data:/var/lib/mysql
environment:
  MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_root_password
  MYSQL_DATABASE: ppsDB
  MYSQL_USER: pps
  MYSQL_PASSWORD_FILE: /run/secrets/db_user_password
secrets:
  - db_root_password
  - db_user_password

wordpress:
  depends_on:
    - db
  image: wordpress:latest
  volumes:
    - wordpress_data:/var/www/html
  ports:
    - "8000:80"
  environment:
    WORDPRESS_DB_HOST: db
    WORDPRESS_DB_NAME: ppsDB # si no se define creo toma el valor wordpress
    WORDPRESS_DB_USER: pps
    WORDPRESS_DB_PASSWORD_FILE: /run/secrets/db_user_password
  secrets:
    - db_user_password

secrets:
  db_user_password:
    file: db_user_password.txt
  db_root_password:
    file: db_root_password.txt

volumes:
  db_data:
  wordpress_data:

```

Recuerde que en este caso no se crean los secretos antes del despliegue (no se usa `docker secret create`), sino que se generan de forma automática en el momento del despliegue. Esto se puede observar en la salida del comando `docker stack deploy`:

```

$ docker stack deploy -c compose_wordpress.yml -d demo
Creating network demo_default
Creating secret demo_db_root_password
Creating secret demo_db_user_password
Creating service demo_db
Creating service demo_wordpress
$

```

Puede comprobar los secretos creados:

```

$ docker secret ls
ID          NAME                DRIVER CREATED    UPDATED
39...17    demo_db_root_password 9 m. ago  9 m. ago
kk...zz    demo_db_user_password 9 m. ago  9 m. ago
$

```

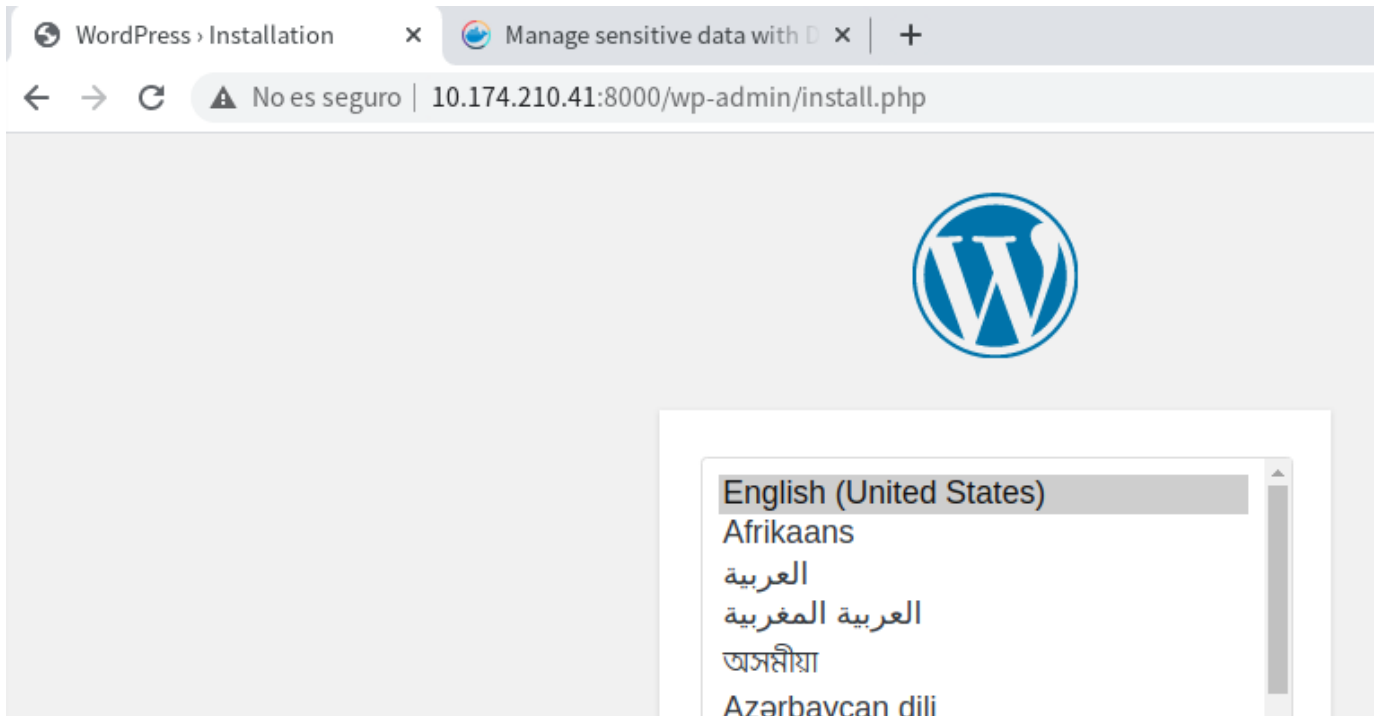
O el estado de los servicios del stack:

```

$ docker stack ls
NAME      SERVICES
demo      2
$ docker stack services demo
ID          NAME      MODE          REPLICAS  IMAGE          PORTS
24hyojjstv53  demo_db  replicated    1/1        mysql:latest
x10j8zw84xkw  demo_wordpress replicated    1/1        wordpress:latest *:8000->80/tcp
usuario@d12-base-01:~/CIBER-PPS/bloque_05_docker/t01-docker-tutorial-101/src_TODO
$

```

Y ahora compruebe si `wordpress` está funcionando de forma conjunta con el servidor `mysql`. Para ello acceda al puerto 8000 de localhost (o a la IP de su equipo) y se debe mostrar la página inicial de configuración del CMS `wordpress`:



Puede hacer pruebas, añadir páginas o artículos, etc.

Para borrar el despliegue `docker stack rm <nombre_deploy>`

```
$ docker stack rm demo
Removing service demo_db
Removing service demo_wordpress
Removing secret demo_db_root_password
Removing secret demo_db_user_password
Removing network demo_default
$
```

Y para borrar el cluster `docker swarm leave`. Tenga en cuenta que en ese caso se eliminan los secretos.

2.3.2 2. docker config

Si la información que utiliza no es "sensible" es posible usar `docker config` en vez de `docker secrets`. En el despliegue previo se podría haber usado para almacenar el usuario y base de datos.

Tiene un ejemplo en este enlace: [Use Swarm Managed Configs](#)

2.3.3 3. Docker compose con build de una imagen

En este [enlace](#) se muestra un ejemplo de un fichero docker-compose que además realiza un build de la imagen que va a desplegar.

Para ello es necesario tener previamente definido el fichero Dockerfile que define la construcción de esa imagen. En el ejemplo tiene disponible el código de la aplicación, el fichero `Dockerfile` y el fichero `docker-compose.yml`.

La parte que nos interesa es como indicarle que tiene que usar la imagen que genera el Dockerfile. Y eso se indica con la clave `build:` dentro de la definición del servicio `web`:

```
services:
  web:
    build: .
    ports:
      - "8000:5000"
  ...
```

En este caso sólo se le indica a build el contexto y busca el fichero `Dockerfile`. Existen más opciones para indicar el nombre del fichero `Dockerfile`, darle nombre a la imagen, etc.

Puede crear la imagen con el comando `docker compose build` que recorre los servicios buscando que imágenes debe construir. También puede invocar `docker compose` como habitualmente y si no existe la imagen la crea en ese momento.

2.3.4 4. Mejorando compose de postgres: iniciar la BD

A los ficheros compose que usan bases de datos se les puede añadir:

- Persistencia a los datos con el uso de un volumen con nombre para la base de datos. Esto ya se ha incluido en el ejemplo de wordpress con un volumen para la base de datos y otro para wordpress.
- Incluir scripts SQL **que cuando se crea el volumen asociado a la base de datos puede crear e iniciar tablas**. Para eso es necesario montar el fichero (o un directorio si hay varios scripts) en el directorio `/docker-entrypoint-initdb.d/` del contenedor.

El fichero `compose_postgres.v3.yml` quedaría:

```
version: '3.1'
services:
  db:
    image: postgres
    environment:
      POSTGRES_PASSWORD: ppspassword
      POSTGRES_USER: pps
      POSTGRES_DB: ppsDB
    volumes: # Mejoras al fichero
      - ./inicia.sql:/docker-entrypoint-initdb.d/inicia.sql
      - db_data:/var/lib/postgresql/data

  adminer:
    image: adminer
    ports:
      - 8080:8080

volumes: # necesario al añadir volumen
  db_data:
```

Y un fichero `inicia.sql` podría ser:

```
CREATE TABLE "test" (
  "id" serial NOT NULL,
  PRIMARY KEY ("id"),
  "nombre" character varying(32) NOT NULL,
  "email" character varying(254) NOT NULL
);

INSERT INTO "test" ("nombre", "email")
VALUES
  ('Ana', 'ana@local.com'),
  ('Raul', 'raul@local.com'),
  ('Raquel', 'raquel@local.com');
```

Al invocar el compose y crear el volumen se ejecuta el script en la base de datos `ppsDB`. La próxima vez que levante el servicio si el volumen existe no se ejecuta el script sql.

2.3.5 5. Anexo - Crear un registro docker en local

En vez de usar DockerHub puede mantener un registro propio para distribuir sus imágenes en local,

```
$ docker service create --name registry --publish published=5000,target=5000 registry:2
$ docker service ls
wbqfqi6e93vgatef1pbklmggm
overall progress: 1 out of 1 tasks
1/1: running [=====]
verify: Service converged
$ docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
getting-started     latest      81....6c     45 minutes ago 223MB
registry            <none>     67....fa     2 months ago  26.2MB
$
```

Usaremos una de las imágenes ya descargadas para añadirla al registro local. Primero debe etiquetar la imagen con el nombre de la máquina (o IP) y puerto del servicio (*when the first part of the tag is a hostname and port, Docker interprets this as the location of a registry, when pushing*).

```
$ docker tag getting-started:latest localhost:5000/getting-started
$ docker images
REPOSITORY          TAG         ID          CREATED        SIZE
getting-started     latest      81...c     51 minutes ago 223MB
localhost:5000/getting-started latest      81...c     51 minutes ago 223MB
```



```
registry          <none> 67..a 2 months ago 26.2MB
$
```

Hacemos *push* a nuestro servicio de registro:

```
$ docker push localhost:5000/getting-started
Using default tag: latest
The push refers to repository [localhost:5000/getting-started]
ed3bd20764dc: Pushed
...
0fcbbeeb0d7: Pushed
latest: digest: sha256:9d0cd8fbbe...aae70b59dee30da4 size: 1789
$
```

Borramos la imagen local:

```
$ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
registry <none> 678dfa38fcfa 2 months ago 26.2MB
$
```

Y hacemos una prueba de descarga desde el registro creado indicando el nombre y puerto del servicio:

```
$ docker pull localhost:5000/getting-started
Using default tag: latest
latest: Pulling from getting-started
...
Digest: sha256:9d0cd8fbbe871...40ba5545a16924aae70b59dee30da4
Status: Downloaded newer image for localhost:5000/getting-started:latest
localhost:5000/getting-started:latest
$
```

Para eliminar el servicio de registro usamos `docker service rm`

```
$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
wbqfqi6e93vg registry replicated 1/1 registry:2 *:5000->5000/tcp
$ docker service rm registry
registry
$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
$ docker pull localhost:5000/getting-started
Using default tag: latest
Error response from daemon: Get http://localhost:5000/v2/: dial tcp 127.0.0.1:5000: connect: connection refused
$
```

Más información: <https://docs.docker.com/registry/deploying/>

3. Servidores

3.1 Servidores Web

Una vez que conoce el funcionamiento básico de la tecnología de contenedores puede desplegar servicios (servidores web, bases de datos, gestores de contenidos, etc.) de una forma rápida y sencilla.

3.1.1 1. Uso básico

Puede probar a desplegar un servidor web nginx o apache (httpd).

```
alu@xd13:~/DW/apache$ docker run -d -p 80:80 --name web httpd
Unable to find image 'httpd:latest' locally
latest: Pulling from library/httpd
...
Digest: sha256:ca375ab8ef2cb8bede6b1bb97a943cce7f0a304d5459c085235b47bc2dccb98cd
Status: Downloaded newer image for httpd:latest
04464ccf11e5fa9b3e809eb0f23fc32f6737d825a29c49cf60497a8570bd2758
alu@xd13:~/DW/apache$
```

Y para probar basta usar `curl` y comprobar que se muestra la página por defecto del servidor Apache:

```
alu@xd13:~/DW/apache$ curl localhost
<html><body><h1>It works!</h1></body></html>
alu@xd13:~/DW/apache$
```

Puede modificar el fichero `/etc/hosts` para "simular" resolución de nombres y crear un dominio `sitio1.internal` (sobre el uso de TLDs privados).

Edite y añada una línea en dicho fichero similar a `127.0.0.1 sitio1.internal`

```
alu@xd13:~/DW/apache$ sudo nano /etc/hosts
[sudo] contraseña para alu:
alu@xd13:~/DW/apache$ cat /etc/hosts
127.0.0.1 sitio1.internal

127.0.0.1 localhost
127.0.1.1 xd13
...
alu@xd13:~/DW/apache$
```

Y ahora puede usar el nombre del dominio "creado":

```
alu@xd13:~/DW/apache$ curl sitio1.internal
<html><body><h1>It works!</h1></body></html>
alu@xd13:~/DW/apache$ firefox sitio1.internal &
```

3.1.2 2. Con sitio web personalizado

Para desplegar un sitio web desarrollado por usted debe consultar en la documentación de DockerHub donde debe copiar (o montar) su sitio web. En el caso de la imagen `httpd` es el directorio `/usr/local/apache2/htdocs/`.

Para ello cree un directorio de nombre `html`, y dentro coloque un sitio web de prueba con un fichero `index.html` y un fichero de estilos CSS:

```
alu@xd13:~/DW/apache$ mkdir html
alu@xd13:~/DW/apache$ code html
# Se crea fichero html y hoja de estilos
alu@xd13:~/DW/apache$ ls -R html
html:
css index.html

html/css:
estilos.css
alu@xd13:~/DW/apache$
```

Para ejecutar y probar basta hacer un bind mount de `html` a `/usr/local/apache2/htdocs/` añadiendo la opción `-v ./html:/usr/local/apache2/htdocs/`:

```
alu@xd13:~/DW/apache$ docker run -d -p 80:80 --name web -v ./html:/usr/local/apache2/htdocs/ httpd
4e78397f71b1151f7bfff33281fbcfac976c6d97ee12cc5999008e596aac7ff98
alu@xd13:~/DW/apache$ curl sitio1.internal
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Apache y Docker</title>
  <link rel="stylesheet" href="css/estilos.css">
</head>
<body>
  <h1>Sitio web Apache personalizado</h1>
</body>
</html>alu@xd13:~/DW/apache$
```

Y para mejorar esta solución puede crear un fichero compose:

compose.yml

```
services:
  apache:
    image: httpd:2.4
    container_name: miweb
    ports:
      - "80:80"
    volumes:
      - ./html:/usr/local/apache2/htdocs
    restart: always
```

3.1.3 3. Imagen con aplicación

En ese caso necesita un fichero Dockerfile que a partir de la imagen de apache copia su web en `/usr/local/apache2/htdocs`:

Dockerfile

```
FROM httpd:2.4
COPY ./html /usr/local/apache2/htdocs
```

```
alu@xd13:~/DW/apache$ ls
docker-compose.yml Dockerfile html
alu@xd13:~/DW/apache$ docker build -t miapp .
[+] Building 2.4s (7/7) FINISHED          docker:default
=> [internal] load build definition from Dockerfile          0.1s
=> => transferring dockerfile: 90B                          0.0s
=> [internal] load metadata for docker.io/library/httpd:2.4  1.8s
=> [internal] load .dockerignore                             0.1s
=> => transferring context: 2B                                0.0s
=> CACHED [1/2] FROM docker.io/library/httpd:2.4@sha256:ca375ab8ef 0.0s
=> [internal] load build context                             0.1s
=> => transferring context: 504B                              0.0s
=> [2/2] COPY ./html /usr/local/apache2/htdocs              0.1s
=> exporting to image                                       0.2s
=> => exporting layers                                       0.1s
=> => writing image sha256:e77f76dd6733523aa6b18b1f37bd9c25ba48aad 0.0s
=> => naming to docker.io/library/miapp                     0.0s
alu@xd13:~/DW/apache$
```

Y ya puede lanzar su propia imagen con su sitio web incorporado:

```
alu@xd13:~/DW/apache$ docker run -d -p 80:80 --name miweb miapp:latest
2cc9f6cbb96fd7a89e847dc635f88b9b48ba7f6895aff3b0dbc6edb401b9300
alu@xd13:~/DW/apache$ curl sitio1.internal
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Apache y Docker</title>
  <link rel="stylesheet" href="css/estilos.css">
</head>
<body>
  <h1>Sitio web Apache personalizado</h1>
</body>
</html>alu@xd13:~/DW/apache$
```

3.2 Bases de Datos

Una vez que conoce el funcionamiento básico de la tecnología de contenedores puede desplegar servicios (servidores web, bases de datos, gestores de contenidos, etc.) de una forma rápida y sencilla.

3.2.1 1. Desde CLI

En este apartado va a desplegar un contenedor usando la imagen de `mysql`. Si consulta la documentación de dicha imagen verá que la base de datos se puede configurar a través del uso de variables de entorno. En este ejemplo inicial se utilizará la variable de entorno `MYSQL_ROOT_PASSWORD` para indicarle al contenedor la clave del usuario root.

Si usa la línea de comandos las variables de entorno se pasan con la opción `-e`, en este caso `-e MYSQL_ROOT_PASSWORD=miclave`.

```
docker run -d --name mibd -e MYSQL_ROOT_PASSWORD=miclave mysql
```

Para comprobar que la BD está operativa se inicia una shell en el contenedor con el comando `docker exec` y dentro se ejecuta el cliente `mysql`:

```
alu@xd13:~/DW$ docker exec -it mibd bash
bash-5.1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 9.4.0 MySQL Community Server - GPL
...

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.015 sec)

mysql>
```

O todo en uno ejecutando el cliente `mysql` desde `docker exec`:

```
alu@xd13:~/DW$ docker exec -it mibd mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 9.4.0 MySQL Community Server - GPL
...

mysql>
```

3.2.2 2. Con compose

De una forma similar a los servidores web puede crear un fichero `compose` y en este caso pasar la información de las variables de entorno:

compose.yaml

```
# Use root/example as user/password credentials
services:
  db:
    image: mysql
    container_name: mibd
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: example
    # (this is just an example, not intended to be a production configuration)
```

3.2.3 3. Con gestor web

Puede añadir al `compose` anterior el servicio `adminer` (o `phpmyadmin`):

```
...
adminer:
  image: adminer
  restart: always
  ports:
    - 8080:8080
```

3.2.4 4. Persistencia

Y para terminar tenga en cuenta la persistencia de los datos, necesita volúmenes para mantener la información [Where to Store Data](#)

3.3 Servidor de Aplicaciones

3.3.1 1. ¿Qué es un servidor de aplicaciones?

Un servidor de aplicaciones es un software que proporciona un entorno para ejecutar aplicaciones, generalmente en el lado del servidor. Sus funciones incluyen:

- Gestionar peticiones HTTP desde clientes (navegadores).
- Ejecutar código Java, servlets, JSP o frameworks web.
- Ofrecer servicios comunes: seguridad, gestión de sesiones, conexión a bases de datos, logging, etc.

👉 Ejemplos de servidores de aplicaciones:

- Apache Tomcat (muy usado para Java EE o Jakarta EE Web Profile)
- WildFly (antes JBoss)
- GlassFish / Payara
- Jetty

3.3.2 2. Apache Tomcat

Tomcat es un servidor open source desarrollado por la Apache Software Foundation. Implementa las especificaciones Servlet y JSP.

Características principales:

- Ligero y fácil de configurar.
- Ideal para desplegar aplicaciones Java web (archivos `.war`).
- Se puede ejecutar como servicio o en contenedor Docker.
- Accesible por defecto en el puerto 8080.

3.3.3 3. Tomcat con Docker

En lugar de instalar Tomcat manualmente, podemos usar una imagen Docker (`tomcat:10` o `tomcat:9`). Ventajas:

- No hay que configurar Java ni Tomcat en el sistema.
- Entorno limpio, reproducible y aislado.
- Ideal para entornos educativos y despliegue rápido.

Requisitos previos

- Tener Docker instalado y funcionando,
- Disponer de una aplicación web Java empaquetada en un archivo `.war`.

3.3.4 4. Práctica

4.1. La aplicación

Crear un directorio:

```
mkdir tomcat-docker
cd tomcat-docker
```

Se le proporciona en este ejemplo una aplicación desarrollada en JSP que solicita que el usuario adivine un nombre. Este es el fichero que debe descargar:

Fichero [guess-number.zip](#)

Ahora basta descomprimir y crear el fichero war:

```
unzip guess-number.zip
cd guess-number
jar cvf guess-number.war *.jsp
```

4.2. Crear la imagen

Usar un Dockerfile similar a este que debe almacenar en la carpeta `tomcat-docker`

Dockerfile

```
# Usar la imagen oficial de Tomcat
FROM tomcat
# FROM tomcat:10-jdk17

# Copiar nuestra app al directorio webapps
COPY ./guess-number/guess-number.war /usr/local/tomcat/webapps/ROOT.war

# Exponer el puerto 8080
EXPOSE 8080
```

Y construir la imagen

```
docker build -t mitomcat-app .
```

4.3. Probarla



Ejecuta en tu terminal

```
docker run -d -p 8080:8080 --name tomcat-app mitomcat-app

firefox http://localhost:8080
```

Deberías ver la aplicación Java funcionando dentro del contenedor Tomcat.

3.3.5 5. Con compose

En este caso no se crea una imagen sino que se usa la de tomcat, y un volumen para la aplicación. Usar un compose similar a este:

compose.yaml

```
services:
  tomcat:
    image: tomcat
    container_name: mitomcat
    ports:
      - "8080:8080"
      - ./guess-number/guess-number.war:/usr/local/tomcat/webapps/ROOT.war
    restart: unless-stopped
```

Y para ejecutar

```
docker compose up -d
```

3.4 Ampliación

Puede ver el curso de OW sobre Apache: [Servidor Web Apache 2.4](#). En el se explican los conceptos de ficheros de configuración, y opciones como virtual hosting, mapeo de URLs, SSL, módulos, etc.

Se muestran a continuación ejemplos de algunas de ellas.

3.4.1 1. Virtual hosts

1.1. Añadir dominios locales al sistema

Edita el archivo `/etc/hosts`

Agregue las siguientes líneas :

```
127.0.0.1    sitio1.local
127.0.0.1    sitio2.local
```

1.2. Estructura directorios

```
mkdir apache-docker
cd apache-docker
```

Y cree esta estructura en su interior. Otra opción es descargar el fichero comprimido:

```
apache-docker/
├── docker-compose.yml
├── html/
│   ├── sitio1/
│   │   └── index.html
│   └── sitio2/
│       └── index.html
└── conf/
    ├── httpd-vhosts.conf
    └── httpd.conf
```

1.3. Configuración apache

Necesita los ficheros:

- `httpd.conf` , configuración genral de apache
- `httpd-vhosts.conf` , configuración de los hosts virtuales `httpd-vhosts.conf`

1.4. Compose

Necesitará un compose similar a este:

```
services:
  apache:
    image: httpd:2.4
    container_name: apache_local
    ports:
      - "80:80"
    volumes:
      # Directorio con el los dos sitios web
      - ../html:/usr/local/apache2/htdocs
      # Ficheros de configuración de apache
      - ../conf/vhosts.conf:/usr/local/apache2/conf/extra/httpd-vhosts.conf
      - ../conf/httpd.conf:/usr/local/apache2/conf/httpd.conf
```

1.5. Probar

```
docker compose up -d
curl sitio1.local
curl sitio2.local
```


3.4.2.2. SSL (autofirmado)

2.1. Entorno

Se añaden a continuación una carpeta `certs` y un nuevo archivo de configuración `vhosts-ssl.conf` y la estructura quedará:

```

apache-docker/
├── docker-compose.yml
├── conf/
│   ├── httpd.conf
│   ├── vhosts.conf
│   └── vhosts-ssl.conf
├── certs/
├── html/
│   ├── sitio1/
│   └── sitio2/

```

2.2. Generar certificados autofirmados

Ejecutar en la carpeta raíz del proyecto:

```

# Crear directorio
apache-docker$ mkdir certs

# Crear Certificado de sitio1
apache-docker$ openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout certs/sitio1.local.key \
-out certs/sitio1.local.crt \
-subj "/C=ES/ST=Madrid/L=Madrid/O=LocalDev/CN=sitio1.local"

# Crear Certificado de sitio2
apache-docker$ openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout certs/sitio2.local.key \
-out certs/sitio2.local.crt \
-subj "/C=ES/ST=Madrid/L=Madrid/O=LocalDev/CN=sitio2.local"

```

2.3. Fichero configuración

Se añade el fichero `vhosts-ssl.conf` del que se muestra un extracto:

```

Listen 443
LoadModule ssl_module modules/mod_ssl.so
LoadModule socache_shmcb_module modules/mod_socache_shmcb.so

<VirtualHost *:443>
    ServerAdmin webmaster@sitio1.local
    ServerName sitio1.local
    DocumentRoot "/usr/local/apache2/htdocs/sitio1"

    SSLEngine on
    SSLCertificateFile "/usr/local/apache2/conf/certs/sitio1.local.crt"
    SSLCertificateKeyFile "/usr/local/apache2/conf/certs/sitio1.local.key"

    <Directory "/usr/local/apache2/htdocs/sitio1">
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog "logs/sitio1-ssl-error.log"
    CustomLog "logs/sitio1-ssl-access.log" common
</VirtualHost>
...

```

Además hay que modificar `httpd.conf` y añadir al final del mismo estas líneas:

```

...
# Incluir también la configuración SSL
Include conf/extra/httpd-vhosts-ssl.conf

```

2.4. Modificar compose

Se modifica el fichero compose para:

- Añadir redirección del puerto 443
- Añadir volúmenes:
- con una ruta a los certificados creados
- con el nuevo fichero de configuración `vhosts-ssl.conf`
-

```
services:
  apache:
    image: httpd:2.4
    container_name: apache_local
    ports:
      - "80:80"
      # SSL
      - "443:443"
    volumes:
      - ./html:/usr/local/apache2/htdocs
      - ./conf/vhosts.conf:/usr/local/apache2/conf/extra/httpd-vhosts.conf
      - ./conf/httpd.conf:/usr/local/apache2/conf/httpd.conf
      # SSL
      - ./conf/vhosts-ssl.conf:/usr/local/apache2/conf/extra/httpd-vhosts-ssl.conf
      - ./certs:/usr/local/apache2/conf/certs
```

2.5. Probar

Tenga en cuenta que al ser certificados autofirmados el navegador no mostrará la página y avisará de un posible fallo de seguridad. Otra posibilidad es usar el comando `curl` con la opción `-k`:

```
$ curl https://sitio2.local
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html
...
$ curl -k https://sitio2.local
<!DOCTYPE html>
...
    <h1>Sitio 2</h1>
</body>
</html>
$
```

Opciones de utilidad con curl:

Opción	Descripción
-L	Sigue las redirecciones.
-k	Ignora errores de certificado.
-I	Muestra solo los encabezados.
-v	Modo detallado (verbose).

2.6. Ampliación

Se le propone modificar la configuración del servidor para que las peticiones al puerto 80 se redirijan al puerto 443 (redirección HTTP a HTTPS)

4. Ejercicios

4.1 Servidores

1. Lanzar contenedor con servidor web nginx. Probar. Parar y borrar.
2. Lanzar contenedor con servidor web nginx mapeando un directorio del host como sitio web (usar por ejemplo fuentes del juego 2048). Probar.
3. Lanzar segundo contenedor nginx usando un puerto diferente. Probar. Parar y borrar.
4. Lanzar contenedor con servidor web apache usando fuente del fichero `Netflix.zip`. Probar. Parar y borrar.
5. Lanzar contenedor con **servidor web con php**. Ver variantes de la imagen php con apache. Mapear un directorio local que contenga un fichero `index.php` de prueba. Probar. Parar y borrar.
6. Lanzar servidor con BD `mysql`. Acceder luego a la consola `mysql` del contenedor usando `docker exec`. Parar y borrar.
7. Lanzar servidor con BD `mariadb` usando un volumen docker. Hacerlo generando una clave de root aleatoria.
8. a. Investigar: Se propone que al lanzar un contenedor con un motor de BD (pe. `mysql`) se cree una base de datos y se rellene con datos iniciales.
Pista: `docker-entrypoint-initdb.d` (puede usar los ficheros de ejemplo).

4.2 Crear su propia imagen

1. Crear una imagen propia con el juego 2028. Antes modificar el `title` de la página principal para indicar vuestro nombre. Probar. Subir a DockerHub.

4.3 Configuraciones multicontenedor

1. Crear un fichero `docker-compose` para lanzar una base de datos y el gestor `adminer` (o `phpmyadmin`) en dos contenedores. Probar el acceso desde `adminer` a la BD (ejemplo en la info de la imagen `adminer`).
2. Modificar anterior para almacenar la BD en un volumen. Generar info desde `adminer` y comprobar que persiste tras reiniciar el contenedor
3. Crear/buscar fichero `docker-compose` para lanzar Wordpress en dos contenedores (aplicación y BD). Probar.
4. Revisar apartado anterior: Añadir una entrada y un tema nuevo a Wordpress. Probar. Detener y borrar los contenedores. Reiniciar el servicio y comprobar que permanece la entrada y el tema. Si no, solucionar.
5. Opcional: probar configuración multicontenedor de algún otro CMS como joomla o drupal.

4.4 Traefik

1. Añadir a ejemplos de clase el servicio `jugar` que lanza la imagen del juego 2048 construida previamente. Debe responder en la dirección `http://jugar.sitio1.internal`.
2. Añadir a ejemplo de clase el CMS Wordpress. Debe responder en la dirección `http://wordpress.midominio.internal`.

5. Traefik

5.1 1. Introducción

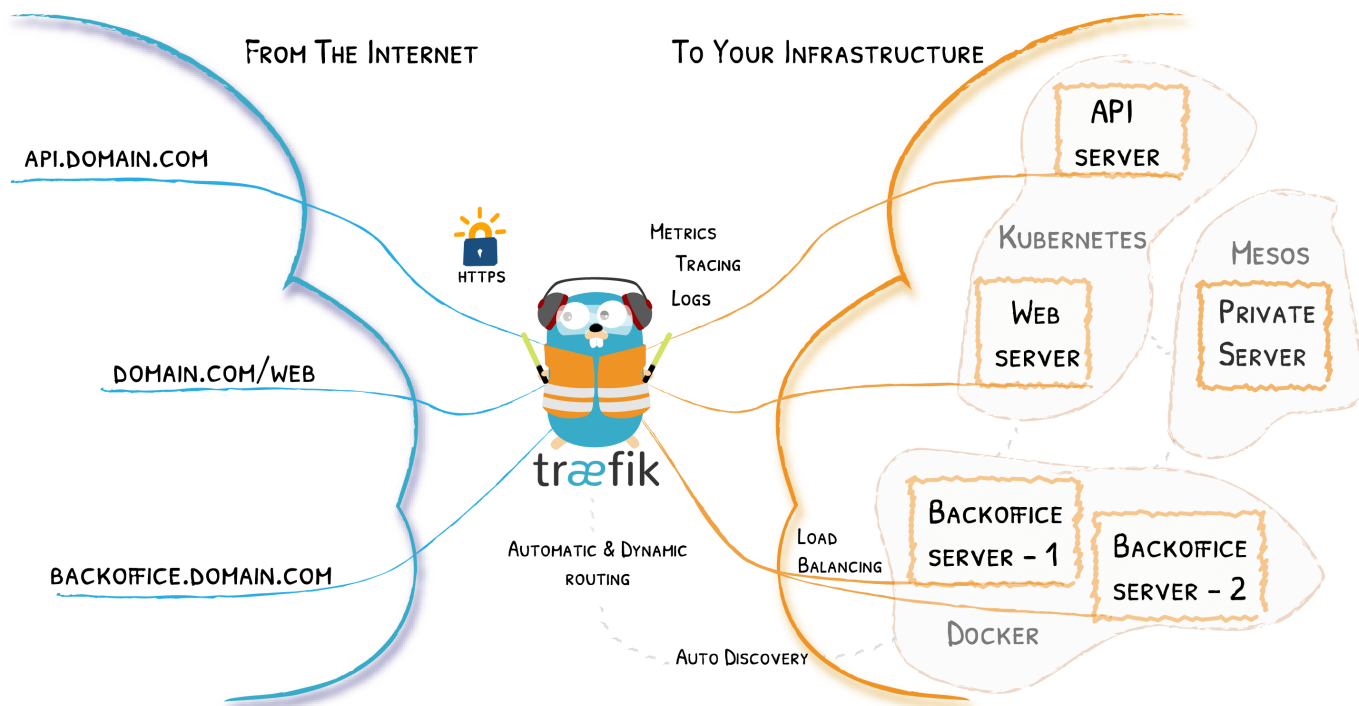
Un proxy inverso es un servidor que se sitúa delante de uno o varios servidores web, e intercepta las solicitudes de los clientes. Esto es diferente de un proxy de reenvío, en el que el proxy se sitúa delante de los clientes. Con un proxy inverso, cuando los clientes envían solicitudes al servidor de origen de un sitio web, el servidor de proxy inverso intercepta esas solicitudes en el perímetro de la red. El servidor proxy inverso enviará entonces las solicitudes al servidor de origen y recibirá las respuestas del servidor de origen.

La diferencia entre un proxy de reenvío y uno inverso es sutil pero importante. Una forma simplificada de resumirla sería decir que un proxy de reenvío se sitúa delante de un cliente y se asegura de que ningún servidor de origen se comunique nunca directamente con ese cliente específico. Por otro lado, un proxy inverso se sitúa delante de un servidor de origen y se asegura de que ningún cliente se comunique nunca directamente con ese servidor de origen.

5.1.1 1.1. Traefik como proxy inverso

Nginx o Apache pueden cumplir perfectamente esta función, aunque hoy en día hay una nueva categoría de proxys reversos pensados para microservicios y también diseñados para “dialogar” con plataformas de contenedores como Docker, Kubernetes, etc. Este es el caso de Traefik.

Traefik is an open-source Edge Router that makes publishing your services a fun and easy experience. It receives requests on behalf of your system and finds out which components are responsible for handling them.



What sets Traefik apart, besides its many features, is that it automatically discovers the right configuration for your services. The magic happens when Traefik inspects your infrastructure, where it finds relevant information and discovers which service serves which request.

The main features include dynamic configuration, automatic service discovery, and support for multiple backends and protocols.

Traefik is based on the concept of EntryPoints, Routers, Middlewares and Services.

- **EntryPoints:** EntryPoints are the network entry points into Traefik. They define the port which will receive the packets, and whether to listen for TCP or UDP.
- **Routers:** A router is in charge of connecting incoming requests to the services that can handle them.
- **Middlewares:** Attached to the routers, middlewares can modify the requests or responses before they are sent to your service
- **Services:** Services are responsible for configuring how to reach the actual services that will eventually handle the incoming requests.

1.1.1. Edge Router¶

Traefik is an Edge Router, it means that it's the door to your platform, and that it intercepts and routes every incoming request: it knows all the logic and every rule that determine which services handle which requests (based on the path, the host, headers, etc.).

1.1.2. Auto Service Discovery¶

Where traditionally edge routers (or reverse proxies) need a configuration file that contains every possible route to your services, Traefik gets them from the services themselves.

Deploying your services, you attach information that tells Traefik the characteristics of the requests the services can handle.

5.2 2. Iniciar Traefik

Se parte del ejemplo que proporciona Traefik en su [guía rápida](#).

El fichero inicial sólo contiene el servicio de nombre `traefik` que lanza Traefik

```
services:
  traefik:
    image: traefik:v3.5
    command:
      - "--api.insecure=true"
```

```

- "--providers.docker=true"
- "--entrypoints.web.address=:80"
ports:
- "80:80"
- "8080:8080"
volumes:
- /var/run/docker.sock:/var/run/docker.sock

```

Inicie el servicio con el comando `docker compose` y puede acceder al *dashboard*:

Because we explicitly enabled insecure mode, the dashboard is reachable on port 8080 without authentication. **Do not enable this flag in production.** You can access the dashboard at: `http://localhost:8080/dashboard/`

5.3 3. Descubriendo servicios

5.3.1 3.1. Ejemplo inicial

La guía rápida nos propone añadir un servicio web de nombre *whoami* que muestra información del equipo sobre el que se ejecuta ese servicio en un fichero de nombre `whoami.yml`.

En Docker, las labels son una forma de añadir metadatos a los contenedores. Traefik usa estas etiquetas para descubrir automáticamente servicios y configurar dinámicamente las rutas, puertos, reglas, etc.

whoami.yml

```

# whoami.yml
services:
  whoami:
    image: traefik/whoami
    labels:
      - "traefik.http.routers.whoami.rule=Host(`whoami.localhost`)"

```

Apply the configuration:

```
docker-compose -f whoami.yml up -d
```

Test Your Setup

```
curl http://whoami.localhost
```

Detener los servicios:

```
docker compose up --remove-orphans -d
```

5.4 Ampliando el ejemplo inicial

Vamos a generar un ejemplo con una dirección "real" y añadir toda la configuración en un único fichero.

Este es el bloque a añadir en `docker-compose.yml`. Observe que se ha modificado el servicio `whoami` para usar el dominio `md.internal`:

```

whoami:
  # A container that exposes an API to show its IP address
  image: traefik/whoami
  labels:
    - "traefik.enable=true" # es necesario si la opción de traefik --providers.docker.exposedbydefault=false
    - "traefik.http.routers.whoami.entrypoints=http" # valor por defecto, se podrá obviar
    - "traefik.http.routers.whoami.rule=Host(`whoami.md.internal`)"

```

Y se ha añadido al ejemplo de la guía rápida las labels:

- `"traefik.enable=true"` : es una opción necesaria en el servicio si en la opciones de traefik estuviera activa la opción `--providers.docker.exposedbydefault=false`. Por defecto cuando Traefik se conecta al socket Docker expone automáticamente todos los contenedores como servicios. Es decir, si `--providers.docker.exposedbydefault` está en `true` (valor por defecto), cualquier contenedor puede ser accesible públicamente si tiene un puerto abierto. Con `traefik.enable` se controla dicha exposición a nivel de servicio.
- `"traefik.http.routers.whoami.entrypoints=web"`. En este caso esta opción no es necesaria ya que `web` y `websecure` son entrypoints ya definidos en Traefik cuando no defines explícitamente otros en la configuración (web se corresponde con el puerto 80).

If not specified, HTTP routers are attached to all entry points.

the entryPoints web (port 80), websecure (port 443), traefik (port 8080) and metrics (port 9100) are created by default. The entryPoints web, websecure are exposed by default using a Service.

Para iniciar los servicios

```
alu@xd13:~/pr/traefik$ docker compose up -d
```

Traefik descubre este nuevo servicio y lo añade a su configuración creando una ruta que enlaza la dirección `whoami.md.internal` al servicio.

```
alu@xd13:~/pr/traefik$ curl -H "Host:whoami.md.internal" localhost
Hostname: fc5ab2a1c07a
IP: 127.0.0.1
...
X-Real-IP: 172.18.0.1
alu@xd13:~/pr/traefik$
```

Observe que se prueba el servicio: `*` con el comando `curl * el parámetro -H "Host:whoami.md.internal"` (ya que la dirección `whoami.md.internal` no tiene asociada ninguna IP añadimos el campo `Host` en la petición HTTP) `*` como destino `localhost`: si intentamos acceder a la dirección `whoami.md.com` (en el navegador o haciendo ping) dará error ya que no existe ese host ni ese dominio y no hay ninguna IP asociada.

Para resolver la IP de un nombre de host se consulta primero el fichero `/etc/hosts` y si no hay correspondencia a los servidores DNS. Para hacer una prueba más cercana a la realidad se asocia en ese fichero el nombre `whoami.md.internal` a la dirección local `127.0.0.1`, Y ya podría usar ese nombre en sus pruebas en local con el navegador, etc.

```
alu@xd13:~/pr/traefik$ sudo nano /etc/hosts
alu@xd13:~/pr/traefik$ cat /etc/hosts
127.0.0.1      whoami.md.internal localhost
127.0.1.1      xd13
...
alu@xd13:~/pr/traefik$
```

Puede probar ahora con `ping` la resolución del nombre a IP:

```
alu@xd13:~/pr/traefik$ ping whoami.md.internal
PING whoami.md.internal (127.0.0.1) 56(84) bytes of data.
64 bytes from whoami.md.internal (127.0.0.1): icmp_seq=1 ttl=64 time=0.064 ms
64 bytes from whoami.md.internal (127.0.0.1): icmp_seq=2 ttl=64 time=0.088 ms
^C
--- whoami.md.internal ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1021ms
rtt min/avg/max/mdev = 0.064/0.076/0.088/0.012 ms
alu@xd13:~/pr/traefik$
```

Y ahora puede acceder con `curl` sin la opción `-H` (o desde el navegador web)

```
alu@xd13:~/pr/traefik$ curl whoami.md.internal
Hostname: 5d2d95b8f9b3
IP: 127.0.0.1
IP: 172.19.0.3
...
alu@xd13:~/pr/traefik$ firefox whoami.md.internal &
[1] 9050
alu@xd13:~/pr/traefik$
```

Añada ahora otras dos direcciones (que se usan en apartados posteriores) al fichero `/etc/hosts`:

- `web.md.internal` (se asociará a un servidor nginx)
- `php.otrodominio.internal` (se asociará a un servidor PHP)

```
alu@xd13:~/pr/traefik$ sudo nano /etc/hosts
alu@xd13:~/pr/traefik$ cat /etc/hosts
127.0.0.1      php.otrodominio.internal web.md.internal whoami.md.internal localhost
127.0.1.1      vm
...
alu@xd13:~/pr/traefik$
```

5.4.1 3.2. Servicio web nginx

En este apartado se añade un servidor web `nginx`. En este caso se asocia el host `web.md.internal` al servicio. El bloque a añadir en `docker-compose.yml` sería:

```
nginx:
  image: nginx
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.nginx.entrypoints=web"
    - "traefik.http.routers.nginx.rule=Host(`web.md.internal`)"
```

Observe que hay que darle un nombre a la "ruta", y se ha usado en las labels `entrypoints` y `rule` el mismo nombre del servicio. Para activar dicho servicio:

```
alu@xd13:~/pr/traefik$ docker compose up -d nginx
[+] Running 1/1
✓ Container p04_traefik-nginx-1 Started 0.0s
alu@xd13:~/pr/traefik$
```

Nuevamente Traefik descubre el servicio en tiempo real y lo añade a sus rutas. Ahora puede acceder a la dirección `web.md.internal` (con curl o firefox)

```
alu@xd13:~/pr/traefik$ curl web.md.internal
<!DOCTYPE html>
...
<body>
  <p>Página inicio servidor nginx. </p>
</body>
</html>
alu@xd13:~/pr/traefik$
```

5.4.2 3.3. Servidor php

En este apartado se añade un servidor web con PHP. Además se usa un TLD distinto (`otrodominio.internal`). Y como variación el servidor web-PHP mapea un directorio local al contenedor. En el directorio está el fichero `index.php` que muestra la IP y nombre del host. El contenido del fichero `index.php` es:

```
<?php echo 'Server IP: ' . $_SERVER['SERVER_ADDR'] . nl2br(" \n");?>
<?php echo 'Hostname: ' . gethostname() . nl2br(" \n");?>
```

Se añade el nuevo servicio a `docker-compose.yml`. Observe el nombre de las rutas usado (`phpserver`) y el valor de `Host`:

```
...
phpserver:
  image: php:apache
  volumes:
    - ./html:/var/www/html
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.phpserver.entrypoints=web"
    - "traefik.http.routers.phpserver.rule=Host(`php.otrodominio.internal`)"
```

Se lanza el servicio:

```
alu@xd13:~/pr/traefik$ docker compose up -d phpserver
[+] Running 1/1
✓ Container p04_traefik-phpserver-1 Started 0.1s
alu@xd13:~/pr/traefik$
```


Y pruebe el nuevo servicio (en consola y navegador):

```
alu@xd13:~/pr/traefik$ curl php.otrodominio.internal
Server IP: 172.20.0.5 <br />
Hostname: 93206dc48cea <br />
alu@xd13:~/pr/traefik$ firefox php.otrodominio.internal &
```

5.5 4. Escalar el servicio: balanceo de carga

Puede "escalar" este último servicio creando replicas con la opción `--scale`:

```
alu@xd13:~/pr/traefik$ docker compose up -d --scale phpserver=3
[+] Running 6/6
✓ Container p04_traefik-reverse-proxy-1 Running 0.0s
✓ Container p04_traefik-nginx-1 Running 0.0s
✓ Container p04_traefik-whoami-1 Running 0.0s
✓ Container p04_traefik-phpserver-1 Running 0.0s
✓ Container p04_traefik-phpserver-3 Started 0.0s
✓ Container p04_traefik-phpserver-2 Started 0.1s
alu@xd13:~/pr/traefik$
```

Si accede ahora al servicio puede comprobar cómo responden las distintas replicas ya que traefik hace balanceo de carga:

```
alu@xd13:~/pr/traefik$ curl php.otrodominio.internal
Server IP: 172.20.0.5 <br />
Hostname: 93206dc48cea <br />
alu@xd13:~/pr/traefik$ curl php.otrodominio.internal
Server IP: 172.20.0.6 <br />
Hostname: a23b9ac33c77 <br />
alu@xd13:~/pr/traefik$ curl php.otrodominio.internal
Server IP: 172.20.0.7 <br />
Hostname: 8ecfc43015ab <br />
alu@xd13:~/pr/traefik$ curl php.otrodominio.internal
Server IP: 172.20.0.5 <br />
Hostname: 93206dc48cea <br />
alu@xd13:~/pr/traefik$
```

5.6 5. Ejercicio propuesto

Añadir a traefik un servicio `Wordpres` en el dominio `miblog.example.internal`. Tenga en cuenta que son dos servicios: `wordpress` y la base de datos.

5.7 6. Enlaces

- [Quickstart de Traefik](#)
- [Traefik Docker Example and Introduction](#)

5.8 7. Anexo: Fichero `docker-compose.yml` completo

```
# compose.yml
services:

  traefik:
    image: traefik:v3.5
    command:
      - "--api.insecure=true"
      - "--providers.docker=true"
      - "--entrypoints.web.address=:80"
    ports:
      - "80:80"
      - "8080:8080"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock

  whoami:
    # A container that exposes an API to show its IP address
    image: traefik/whoami
    labels:
      - "traefik.enable=true" # es necesario si la opción de traefik --providers.docker.exposedbydefault=false
      - "traefik.http.routers.whoami.entrypoints=web" # valor por defecto, se podrá obviar
      - "traefik.http.routers.whoami.rule=Host(`whoami.md.internal`)"

  nginx:
```

```
image: nginx
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.nginx.entrypoints=web"
  - "traefik.http.routers.nginx.rule=Host(`web.md.internal`)"

phpserver:
image: php:apache
volumes:
  - ./html:/var/www/html
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.phpserver.entrypoints=web"
  - "traefik.http.routers.phpserver.rule=Host(`php.otrodominio.internal`)"
```

Aviso: en este enunciado se usa como TLD para los ejemplos `.internal`. Es uno de los dominios recomendados para *Private DNS Namespaces*, entre los que podría usar `.intranet`, `.private`, `.corp`, `.home` o `.lan`. En prácticas posteriores se usará un dominio DNS real.

6. Cluster: Docker Swarm

6.1 1. Introducción

Se introduce este tema después de hacer las prácticas de Docker. Puede consultar esta referencia con una explicación de los conceptos de *stack*, *service*, *nodo*, etc: <https://docs.docker.com/engine/swarm/key-concepts/>

En la práctica se creará un cluster de 3 nodos que deben estar en la misma red: uno será el *manager* (gestor) y el resto *workers*. Para el despliegue de los tres nodos puede elegir:

- Opción 1: crear tres máquinas virtuales con KVM (o VMware, Dropbox) e instalar docker en ellas.
- Opción 2: crear tres instancias EC2 en AWS
- Opción 3 específica de Ubuntu: usar la utilidad *multipass* para crear los tres nodos e instalar docker en ellos (ver anexo).

En esta práctica se opta por la primera de ellas.

6.2 2. Preparación del swarm

6.2.1 2.1. Nodos

- Se utiliza una máquina virtual ya preparada con Debian sin interfaz gráfico. Lleva instalada `docker`, `sudo`, `curl` y un servidor ssh.
- Como se necesitan tres máquinas virtuales lo más sencillo es preparar una y clonar las otras dos (tiene instrucciones de cómo hacerlo en forma "diferencial" soft o linked en KVM en el anexo).
- Al iniciar las máquinas se modifica el *hostname* para que sea más sencillo identificarlas.
- Se accede mediante ssh desde el *host* a las máquinas virtuales: esto refleja una situación real de trabajo (y además permite copiar y pegar).

6.2.2 2.2. Iniciar el "enjambre" o swarm

Tenga en cuenta que:

- Las IPs de las máquinas virtuales dependerán de la configuración de cada sistema.
- Los usuarios en las máquinas virtuales pueden tener nombres distintos: `clases`, `alu`, `usuario`, etc.

El primer paso es conectarse al nodo manager usando ssh:

```
clases@vm:~$ ssh alu@192.168.125.136
alu@192.168.125.136's password:
Linux manager 6.1.0-13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.55-1 (2023-09-29) x86_64
...
alu@manager:~$
```

El cluster se inicia con el comando `docker swarm init`. Al iniciarlo muestra el *token* de invitación para los nodos tipo *worker*:

```
alu@manager:~$ docker swarm init
Swarm initialized: current node (szbzjfi2mbaiqif01a17u0qjg) is now a manager.

To add a worker to this swarm, run the following command:
    docker swarm join --token SWMTKN-1-0fuvq18x7r6we68t2n584wernrf1nm0n0e3gz90cvaj4qm1a4z-81efbc8r8qxwa7zhm2u1ytpv0 192.168.125.136:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
alu@manager:~$
```

Una vez iniciado puede comprobar con el comando `docker node ls` que el único nodo existente es el *manager*, que aparece como activo y con estado `Leader`

```
alu@manager:~$ docker node ls
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER
szbzjfi2mbaiqif01a17u0qjg * manager        Ready     Active           Leader
alu@manager:~$
```

6.2.3 2.3. Unir nodos *worker* al "enjambre"

Para unir un nodo al cluster o enjambre es necesario usar un *token* de invitación. El *token* se muestra:

- al iniciar el cluster en el manager
- o ejecutando en el manager el comando `docker swarm join-token worker`.

Por lo tanto para añadir un *worker* al cluster es necesario conectarse por ssh al nuevo nodo y una vez conectado ejecutar el comando

```
docker swarm join:
```

```
alu@worker-01:~$ docker swarm join --token SWMTKN-1-0fuvq18x6r6we68t2n584wernrf1nm0n0e3gz90cvaj4qm1a4z-81efbc8r8qxa7zhm2ulytpv0 192.168.125.136:2377
This node joined a swarm as a worker.
alu@worker-01:~$
```

En el nodo *manager* puede comprobar que el nodo se ha añadido con `docker node ls`:

```
alu@manager:~$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER
9nhxurspobgirwh5m1d5h0fvf *	manager	Ready	Active	Leader
zm7acf20ysqblhalyeicrse6w	worker-01	Ready	Active	

```
alu@manager:~$
```

6.3 3. Desplegar servicio

6.3.1 3.1. Despliegue manual

Para probar el cluster **se crea en el *manager*** un servicio que llamaremos *helloworld* usando la imagen `repocurso/helloworld-node-microservice`. Esta imagen es una aplicación web que muestra el nombre (id del contenedor) del equipo.

Observe cómo se crea el servicio (en este caso se hace con una única replica) con el comando `docker service create`:

```
alu@manager:~$ docker service create --replicas 1 --name helloworld -p 8080:8080 repocurso/helloworld-node-microservice
zzcmp70yivslrzdghqyxhjblyl
overall progress: 0 out of 1 tasks
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
alu@manager:~$
```

Para probar el servicio use el comando `curl` (en vez de un navegador). Desde el *manager*:

```
alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "8e088d6016c9".
alu@manager:~$
```

O desde el *worker*

```
alu@worker-01:~$ curl 127.0.0.1:8080
Hello World from host "8e088d6016c9".
alu@worker-01:~$
```

Puede acceder también desde cualquiera de los dos nodos con la IP de los nodos, y en todos los casos se accede correctamente al servicio aunque el contenedor esté sólo en uno de los nodos:

```
alu@manager:~$ curl 192.168.125.136:8080
Hello World from host "692f6380f27b".
alu@manager:~$ curl 192.168.125.141:8080
Hello World from host "692f6380f27b".
alu@manager:~$
```

Si añade otro nodo al enjambre (se conecta al nuevo nodo por ssh y ejecuta el comando `docker swarm join`):

```
alu@worker-02:~$ docker swarm join --token SWMTKN-1-0fuvq18x6r6we68t2n584wernrf1nm0n0e3gz90cvaj4qm1a4z-81efbc8r8qxa7zhm2ulytpv0 192.168.125.136:2377
This node joined a swarm as a worker.
alu@worker-02:~$
```

Puede comprobar en el *manager* cómo se ha añadido el nodo:

```

alu@manager:~$ docker node ls
ID                HOSTNAME        STATUS    AVAILABILITY  MANAGER STATUS
9nhxurspobgirwh5md5h0fvf * manager        Ready     Active         Leader
zm7acf20ysqblhalyeicrse6w worker-01       Ready     Active
s53vme3dp6tr2jvjmfr88avpt worker-02       Ready     Active
alu@manager:~$

```

Ahora puede **escalar** a cuatro réplicas el servicio usando el comando `docker service scale` e indicando el nombre del servicio y el número de réplicas:

```

alu@manager:~$ docker service scale helloworld=4
helloworld scaled to 4
overall progress: 4 out of 4 tasks
1/4: running [=====]
2/4: running [=====]
3/4: running [=====]
4/4: running [=====]
verify: Service converged
alu@manager:~$

```

Se puede observar a continuación que las réplicas se distribuyen en los tres nodos, y responden a las peticiones según un algoritmo *round-robin*. Haga varias peticiones y observe cómo va cambiando el identificador del contenedor:

```

alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "eb5da9711349".
alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "acb9bc2b68ba".
alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "1f4f0b7c2822".
alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "8e088d6016c9".
alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "eb5da9711349".
alu@manager:~$

```

Hasta ahora se han realizado las pruebas desde los nodos del cluster, pero puede acceder al servicio desde el host usando la IP de cualquiera de los nodos del cluster:

```

clases@vm:~$ curl 192.168.125.141:8080
Hello World from host "eb5da9711349".
clases@vm:~$ curl 192.168.125.141:8080
Hello World from host "1f4f0b7c2822".
clases@vm:~$ curl 192.168.125.141:8080
Hello World from host "acb9bc2b68ba".
clases@vm:~$ curl 192.168.125.141:8080
Hello World from host "8e088d6016c9".
clases@vm:~$ curl 192.168.125.141:8080
Hello World from host "eb5da9711349".
clases@vm:~$

```

Puede comprobar la distribución del servicio entre los nodos con el comando `docker service ps <nombreServicio>`. En este caso cada nodo "corre" un contenedor y uno de ellos dos.

```

alu@manager:~$ docker service ps helloworld

```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
z1phnld17fcz	helloworld.1	repocurso/helloworld-node-microservice:latest	manager	Running	Running 6 minutes ago		
pa5hp3zsudf3	helloworld.2	repocurso/helloworld-node-microservice:latest	worker-02	Running	Running about a minute ago		
pppd9mdjj06c	helloworld.3	repocurso/helloworld-node-microservice:latest	worker-01	Running	Running about a minute ago		
or8ythvqgb8q	helloworld.4	repocurso/helloworld-node-microservice:latest	worker-01	Running	Running about a minute ago		

```

alu@manager:~$

```

6.3.2.3.2. Borrar el servicio

Para eliminar el servicio desplegado se usa el comando `docker service rm <nombreServicio>`

```

alu@manager:~$ docker service rm helloworld
helloworld
alu@manager:~$ docker service ls

```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
alu@manager:~\$					

6.3.3 3.3. Desplegar usando stack y fichero compose

Cuando la aplicación a desplegar incluye varios servicios, volúmenes, networks, etc. es recomendable hacer el despliegue desde un fichero que tendrá un formato similar al `docker-compose` de temas anteriores. Este fichero además tendrá etiquetas específicas de un despliegue `swarm` como por ejemplo el número de replicas.

Es este ejemplo se despliega el servicio anterior usando el fichero `helloworld.yml`. Observe los campos `deploy` y `replicas`:

```
version: '3.7'

services:
  helloworld:
    image: repocurso/helloworld-node-microservice
    ports:
      - "8080:8080"
    deploy:
      replicas: 2
```

Para copiar el fichero desde el host hasta el manager se usa el comando `scp fichero usuario@IP:destino`:

```
clases@vm:~$ scp helloworld.yml alu@192.168.125.136:
alu@192.168.125.136's password:
helloworld.yml      100% 160   145.2KB/s   00:00
clases@vm:~$
```

Para desplegar el servicio desde el *manager* se utiliza el comando `docker stack deploy -c <fichero.yml> <nombreStack>`

```
alu@manager:~$ docker stack deploy -c helloworld.yml demo
Creating network demo_default
Creating service demo_helloworld
alu@manager:~$ docker stack ps demo
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
v23bfy99i0yd	demo_helloworld.1	repocurso/helloworld-node-microservice:latest	worker-01	Running	Running 25 seconds ago		
vgz5h3b5gp6s	demo_helloworld.2	repocurso/helloworld-node-microservice:latest	worker-02	Running	Running 25 seconds ago		
ucsa2fut5n3v	demo_helloworld.3	repocurso/helloworld-node-microservice:latest	manager	Running	Running 25 seconds ago		

```
alu@manager:~$
```

Y para eliminar el despliegue se utiliza el comando `docker stack rm <servicio>`:

```
alu@manager:~$ docker stack rm demo
Removing service demo_helloworld
Removing network demo_default
alu@manager:~$
```

6.4 4. Herramienta gráfica: portainer

Si desea visualizar y trabajar con el cluster de forma gráfica una posible opción es usar `portainer`: <https://www.portainer.io/>. Esta aplicación despliega agentes sobre los nodos del cluster.

Para utilizarla descargamos el fichero yaml con una configuración ya preparada desde su página web con el comando `curl -L https://downloads.portainer.io/portainer-agent-stack.yml -o portainer-agent-stack.yml` y lo copiamos con `scp` al manager (o lo puede descargar directamente en el manager)

```
clases@vm:~$ curl -L https://downloads.portainer.io/portainer-agent-stack.yml -o portainer-agent-stack.yml
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           % Dload  % Upload   Total   Spent    Left   Speed
100    791  100    791    0     0    715     0  0:00:01  0:00:01 --:--:--  726
clases@vm:~$ ls
alu@192.168.125.136 helloworld.yml portainer-agent-stack.yml
clases@vm:~$ scp portainer-agent-stack.yml alu@192.168.125.136:
alu@192.168.125.136's password:
portainer-agent-stack.yml      100% 791   1.6MB/s   00:00
clases@vm:~$
```

Y se usa el fichero desde el *manager* para hacer el despliegue:

```
alu@manager:~$ ls
helloworld.yml portainer-agent-stack.yml
alu@manager:~$ docker stack deploy -c portainer-agent-stack.yml portainer
Creating network portainer_agent_network
Creating service portainer_agent
Creating service portainer_portainer
alu@manager:~$
```

Y si observa el despliegue con `docker service ls` hay tantos agentes como nodos y un gestor:

```
alu@manager:~$ docker service ls
ID            NAME          MODE          REPLICAS  IMAGE                                  PORTS
d3ispnqym6s  portainer_agent  global        3/3       portainer/agent:2.11.1
dynd71w6qjxn portainer_portainer  replicated    1/1       portainer/portainer-ce:2.11.1      *:8000->8000/tcp, *:9000->9000/tcp, *:9443->9443/tcp
alu@manager:~$
```

Puede comprobar la distribución con `docker service ps`:

```
alu@manager:~$ docker service ps portainer_portainer
ID            NAME          IMAGE          NODE    DESIRED STATE  CURRENT STATE          ERROR          PORTS
ad941kp4oqjg portainer_portainer.1  portainer/portainer-ce:2.11.1  manager  Running        Running 3 minutes ago
alu@manager:~$ docker service ps portainer_agent
ID            NAME          IMAGE          NODE    DESIRED STATE  CURRENT STATE          ERROR          PORTS
d2wrk62bcgwj portainer_agent.9nhxurspobgirwh5m1d5h0fvf  portainer/agent:2.11.1  manager  Running        Running 3 minutes ago
ur56o8xcj9ja portainer_agent.s53vme3dp6tr2jvjmfr88avpt  portainer/agent:2.11.1  worker-02  Running        Running 3 minutes ago
ke16b2mwa9oy portainer_agent.zm7acf20ysqblhalyeicrse6w  portainer/agent:2.11.1  worker-01  Running        Running 3 minutes ago
alu@manager:~$
```

Acceda ahora usando un navegador **a la IP del manager** (o cualquier otra IP del enjambre) **usando el puerto 9000** y se mostrará el tablero de gestión de portainer. En el primer acceso deberá crear un usuario y clave.

A continuación puede desde el tablero hacer despliegues de. En este ejemplo se realiza el despliegue de `helloworld` desde `portainer` usando el fichero `helloworld.yml`. Para ello acceda al "Dashboard", busque "Stacks", botón "Add Stack" y la opción "Upload from your computer".

La siguiente captura se obtiene desde "Dashboard", enlace "Cluster visualizer".

6.5 5. Anexos

6.5.1 5.2. KVM y clonado suave

1. Partimos de una imagen de Debian 12 con servidor ssh y sin entorno de escritorio. Lleva ya configurado `sudo` e instalado `docker`. Está disponible en la Unidad Compartida.
2. Vamos a hacer un clonado del disco de esa máquina, pero un clonado "suave" o diferencial. La idea es crear 3 máquinas que compartan como base el disco anterior:

```
[clases@d12:~/Descargas/vms]
$ qemu-img create -f qcow2 -b ./sd12-base-clone.qcow2 -o backing_fmt=qcow2 ./manager.qcow2
Formatting './manager.qcow2', fmt=qcow2 cluster_size=65536
...
[clases@d12:~/Descargas/vms]
$ qemu-img create -f qcow2 -b ./sd12-base-clone.qcow2 -o backing_fmt=qcow2 ./worker-01.qcow2
Formatting './worker-01.qcow2', fmt=qcow2 cluster_size=65536
```

```
...
[clases@d12:~/Descargas/vms]
$ qemu-img create -f qcow2 -b ./sd12-base-clone.qcow2 -o backing_fmt=qcow2 ./worker-02.qcow2
Formatting './worker-02.qcow2', fmt=qcow2 cluster_size=65536
...
[clases@d12:~/Descargas/vms]
$ ls -lh worker-* manager.qcow2 sd12-base-clone.qcow2
-rw-r--r-- 1 clases clases 193K oct 25 18:24 manager.qcow2
-rw----- 1 clases clases 3,8G oct 25 18:12 sd12-base-clone.qcow2
-rw-r--r-- 1 clases clases 193K oct 25 18:24 worker-01.qcow2
-rw-r--r-- 1 clases clases 193K oct 25 18:24 worker-02.qcow2
[clases@d12:~/Descargas/vms]
$
```

1. Creados los tres discos, creamos con `virtual manager` 3 máquinas usando los discos creados.
2. Es recomendable al iniciar las máquinas modificar el `hostname` editando los ficheros `/etc/hosts` y `/etc/hostname`.
3. Es preferible conectarse a las máquinas virtuales desde el host usando `ssh`. Anote las IPs de cada una de ellas (e incluso puede modificar el fichero `/etc/hosts` del host para asignarle nombres a las IPs de las máquinas virtuales).

Aviso: es preferible no utilizar el disco base de la clonación.

6.5.2 5.3. Uso de multipass

Aviso: Este anexo no ha sido probado este curso.

En el caso de hacerlo con la herramienta `multipass` de Ubuntu en vez de con máquinas virtuales

```
multipass launch -v -n nodo1 -m 512MGB # -m 1GB
multipass exec nodo1 -- curl -fsSL https://get.docker.com -o get-docker.sh
multipass exec nodo1 -- sudo sh get-docker.sh
```

Nota: evitar la instalación de docker con snap, parece que da problema al desplegar un "stack"

Para ver los nodos creados:

```
$ multipass list
Name      State   IPv4          Image
manager   Running 10.174.210.86 Ubuntu 20.04 LTS 172.17.0.1
nodo1     Running 10.174.210.224 Ubuntu 20.04 LTS 172.17.0.1
nodo2     Running 10.174.210.4  Ubuntu 20.04 LTS 172.17.0.1
$
```

Para iniciar una sesión en un nodo puede usar el comando `multipass shell`

```
$ multipass shell manager
Welcome to Ubuntu 20.04.2 LTS (GNU/Linux 5.4.0-65-generic x86_64)
...
ubuntu@manager:~$
```


7. Fuentes ejemplos o ejercicios

7.1 1. Código ejemplos tutorial

- [Ejemplos Tutorial](#)
- [Ejemplos Ampliación](#)

7.2 2. Código ejemplos de sitios web

- [2048 game](#)
- [Netflix piloto](#)

7.3 3. Código ejemplo tomcat

- [Código Guess Number](#)

7.4 4. Scripts SQL

- [Ejemplos scripts sql](#)

7.5 5. Docker Swarm

- [helloworld.yml](#)
- [portainer-agent-stack.yml](#)

8. Seguridad en Docker

8.1 1. Seguridad en el host

Recomendaciones:

- El host debe estar actualizado: sistema operativo, versión de Docker, bibliotecas, etc.
- Siempre tener en mente minimizar la superficie de ataque.
- Revisar permisos de usuario para acceder al cliente de Docker, evitar el uso de `root` o `sudo`.

8.2 2. Seguridad en el demonio docker

El demonio Docker se ejecuta con privilegios de superusuario. Por ello es importante controlar el acceso a ficheros de configuración de Docker o al propio `socket`.

La localización por defecto del fichero de configuración del demonio `dockerd` en Linux es `/etc/docker/daemon.json` y en Windows `%programdata%\docker\config\daemon.json`. Si el fichero no existe se usan las opciones por defecto. Tiene más información en este enlace sobre este fichero: <https://docs.docker.com/engine/reference/commandline/dockerd/#daemon-configuration-file>

Un ejemplo de fichero `daemon.json`:

```
{
  "debug": true,
  "default-ulimits": {
    "nofile": {
      "Hard": 64000,
      "Name": "nofile",
      "Soft": 64000
    }
  },
  "icc": false,
  ...
}
```

Algunas opciones a controlar:

- `debug`: para activar o desactivar el modo de depuración, debe estar a `false` en producción.
- `default-ulimits`: para limitar el número de descriptores de fichero (se puede establecer también en el momento de lanzar el contenedor).
- `icc`: Controla que la intercomunicación entre contenedores esté limitada (por defecto a `false`).

8.3 3. Seguridad en la ejecución de contenedores

8.3.1 3.1. Limitando recursos en la ejecución

Puede ser conveniente configurar límites de recursos al contenedor. De hecho al lanzar el contenedor se pueden aumentar o disminuir con respecto a los establecidos por defecto en el fichero de configuración.

Por ejemplo, para modificar y limitar en ejecución `ulimit` (el número de ficheros):

```
docker run --rm --ulimit nofile=512:512 debian sh -c "ulimit -n"
```

Otros recursos que se pueden limitar en el contenedor son el uso de CPU, memoria, etc. Vea ejemplos de limitación de CPU o memoria

```
docker run --rm -it --cpus="0.5" debian /bin/bash
docker run --rm -it -m 2GB debian /bin/bash
```

Para ver el uso de memoria puede ejecutar el comando `docker stats`

```
$ docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     ....
```

```
698494b1bb3f brave_jackson 0.00% 2.027MiB / 512MiB 0.40%
$
```

Tiene más información sobre opciones de `docker run` en este enlace: <https://docs.docker.com/engine/reference/commandline/run/>. Entre ellas se puede establecer un reinicio en caso de fallo, y además fijar un límite máximo de reinicios ante fallos con la opción `--restart=on-failure:5`.

8.3.2 3.2. Limitando el usuario y los privilegios

3.2.1. User root

Hay que prestar atención a los privilegios del usuario dentro del contenedor: *por defecto los contenedores se ejecutan como `root`*. Sería recomendable que las aplicaciones dentro del contenedor no se ejecuten como `root` utilizando la opción `-u` al lanzar el contenedor. Vea la diferencia en la ejecución del comando `ls /root` en estos dos casos:

```
$ docker run --rm -u 4400 alpine ls /root -al
ls: cant open '/root': Permission denied
total 0
$ docker run --rm alpine ls /root -al
total 8
drwx----- 2 root root 4096 Apr 14 10:25 .
drwxr-xr-x 1 root root 4096 Apr 27 17:14 ..
$
```

El usuario `root` tiene restringidas muchas capacidades, pero evitar su uso minimiza los posibles riesgos. Si la aplicación en el contenedor no necesita privilegios de `root` puede usar la opción `--user UID:GID` con `docker run`. Tiene un ejemplo de lo que es posible en *Docker containers with root privileges* <https://neoteric.eu/blog/docker-containers-with-root-privileges/>

En un fichero `Dockerfile` esta limitación se consigue con `USER`:

```
FROM ubuntu:latest
#Add a user with userid 8877 and name nonroot
RUN useradd -u 8877 nonroot
#Run Container as nonroot
USER nonroot
....
```

3.2.2. Permisos adicionales

Debe evitar el uso del flag `--privileged` ya que da permiso adicionales al contenedor sobre el host permitiendo todas las *capabilities* de Linux.

En este caso es recomendable la lectura de la regla [RULE #3 - Limit capabilities](#) de *Docker Security Cheat Sheet* de OWASP.

8.3.3 3.3. Acceso al socket de docker desde el contenedor

A veces se usa en sistemas de integración continua (o al ejecutar Docker sobre Docker): *montar el socket de Docker dentro de un contenedor*.

En este ejemplo vea como el contenedor monta como volumen el `socket` de Docker: `-v /var/run/docker.sock:/var/run/docker.sock`. Ya dentro del contenedor con el comando `docker ps` se ven los contenedores que se están ejecutando en el host (fuera del contenedor):

```
$ docker run -v /var/run/docker.sock:/var/run/docker.sock --rm -it docker
# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS          NAMES
db22c6f04cef   docker   "docker-entrypoint.s..." 30 seconds ago   Up 29 seconds   -              practical_fermat
#
```

Observe que desde el host se obtiene la misma salida:

```
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS          NAMES
db22c6f04cef   docker   "docker-entrypoint.s..." 58 seconds ago   Up 57 seconds   -              practical_fermat
ango@tos:/etc/docker$
```

A no ser que sea necesario (hay herramientas que necesitan de esta opción) no lo haga, y si lo hace sólo en entornos controlados con imágenes verificadas como seguras, etc.

Docker socket `/var/run/docker.sock` is the UNIX socket that Docker is listening to. This is the primary entry point for the Docker API. The owner of this socket is root. Giving someone access to it is equivalent to giving unrestricted root access to your host. Do not expose `/var/run/docker.sock` to other containers. If you are running your docker image with `-v /var/run/docker.sock:/var/run/docker.sock` or similar, you should change it. Remember that mounting the socket read-only is not a solution but only makes it harder to exploit. Mounting the Docker daemon socket gives the control of the daemon to the container. However, this process should only be used with trusted containers when necessary.

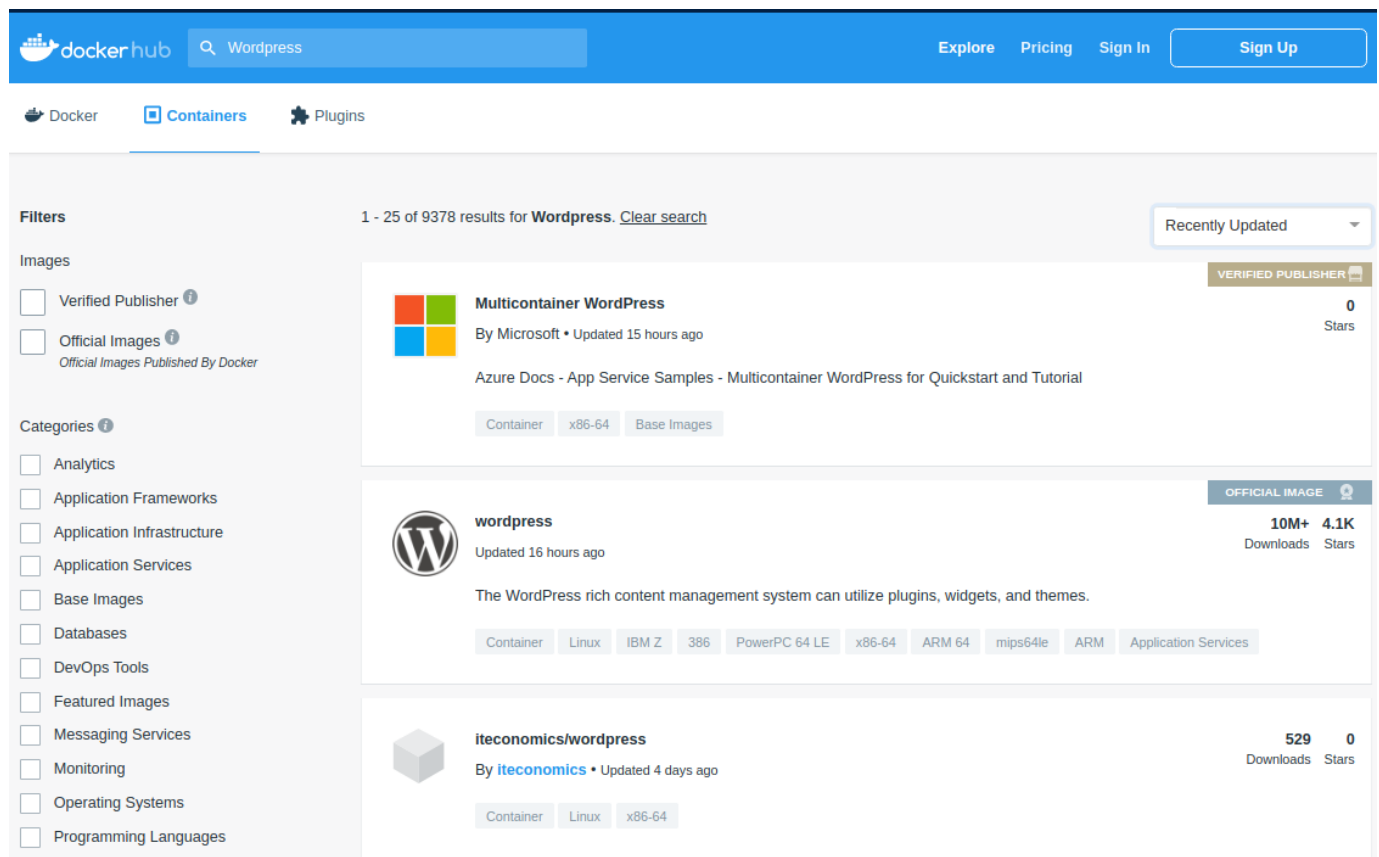
8.4.4. Seguridad en la construcción de imágenes

8.4.1 4.1. Fuentes fiables

En general solo debemos ejecutar contenedores que provengan de fuentes fiables. Y si es posible construirlo partir de un fichero `Dockerfile` lo que permite auditar la imagen.

Por ejemplo, si accede a <http://hub.docker.com> y busca contenedores WordPress salen varias opciones. Puede filtrarlas por valoraciones, descargas, última modificación, etc. Siempre que sea posible debe optar por imágenes **Verified Publisher** y **Official Image**.

En esta figura tiene dos imágenes, una oficial de Docker y otra de una fuente verificada (Microsoft).



8.4.2 4.2. DCT: Docker Content Trust

Se puede asociar un *hash* a cada imagen Docker para verificar que es correcta y no se ha modificado. Esta verificación está desactivada por defecto. Para activarla se configura una variable de entorno, `DOCKER_CONTENT_TRUST` con el valor `1`. Una vez activada cualquier descarga de una imagen implicará la comprobación del *hash*.

Observe en la siguiente ejecución como la descarga de una imagen sin firma no es posible si previamente ha activado `DCT`. Si posteriormente la desactiva será posible la descarga:

```
# se activa DCT
$ export DOCKER_CONTENT_TRUST=1
# Se intenta descargar una imagen sin firmar
$ docker pull rghudok/dct-test:unsigned
No valid trust data for unsigned
# ahora se desactiva
$ export DOCKER_CONTENT_TRUST=0
# Se intenta descargar una imagen sin firmar de nuevo
$ docker pull rghudok/dct-test:unsigned
unsigned: Pulling from rghudok/dct-test
0669b0daf1fb: Pull complete
Digest: sha256:e8d2db0f9cc124273e5f3bbbd2006bf2f3629953983df859e3aa8092134fa373
Status: Downloaded newer image for rghudok/dct-test:unsigned
docker.io/rghudok/dct-test:unsigned
$
```

Prueba ahora la misma secuencia con una imagen firmada:

```
# se activa DCT
$ export DOCKER_CONTENT_TRUST=1
# Se intenta descargar una imagen sin firmar
$ docker run rghudok/dct-test:unsigned
docker: No valid trust data for unsigned.
See 'docker run --help'.
# Se intenta descargar ahora una imagen con firma
$ docker run rghudok/dct-test:signed
Unable to find image 'rghudok/dct-test:signed' locally
docker.io/rghudok/dct-test@sha256:e8d2db0f9cc124273e5f3bbbd2006bf2f3629953983df859e3aa8092134fa373: Pulling from rghudok/dct-test
0669b0daf1fb: Pull complete
Digest: sha256:e8d2db0f9cc124273e5f3bbbd2006bf2f3629953983df859e3aa8092134fa373
Status: Downloaded newer image for rghudok/dct-test@sha256:e8d2db0f9cc124273e5f3bbbd2006bf2f3629953983df859e3aa8092134fa373
Tagging rghudok/dct-test@sha256:e8d2db0f9cc124273e5f3bbbd2006bf2f3629953983df859e3aa8092134fa373 as rghudok/dct-test:signed
Hello this is busybox!
$
```

Tiene una explicación completa del proceso de firma y verificación en el enlace [Docker content trust 101](#). También tiene el proceso disponible en uno de los videos del curso de Docker disponible en la plataforma.

8.5 5. Mejorando el fichero dockerfile

8.5.1 5.1. CMD y ENTRYPOINT

Lo primero sería aclararse con el uso de `CMD` y `ENTRYPOINT`, y como se aplican al lanzar un contenedor. Para ello lo más recomendable es seguir este artículo donde lo explica paso a paso con un ejemplo: [Docker CMD vs. Entrypoint Commands](#) o este en castellano: Docker: [Diferencia entre ENTRYPOINT y CMD](#).

8.5.2 5.2. COPY o ADD

Otro aspecto a revisar es el uso de `COPY` o `ADD`. Otro artículo con un paso a paso explicando su uso y sus diferencias: [Docker ADD vs. COPY: What are the Differences?](#)

Resumen : usar `COPY` siempre que sea posible.

Docker's official documentation notes that COPY should always be the go-to instruction as it is more transparent than ADD.

The Docker team also strongly discourages using `ADD` to download and copy a package from a URL. Instead, it's safer and more efficient to use `wget` or `curl` within a `RUN` command. By doing so, you avoid creating an additional image layer and save space.

Conclusion: To sum up – use `COPY`. As Docker itself suggests, avoid the `ADD` command unless you need to extract a local tar file.

8.5.3 5.3. Multistage

El objetivo es reducir el tamaño de la imagen final realizando todos los pasos previos de compilación, dependencias, etc. en una imagen intermedia. En la imagen final sólo se incorpora el resultado de la fase previa. Para demostrarlo en este ejemplo se usa un pequeño programa en `Go` y un par de ficheros `Dockerfile`. Los detalles están en este artículo: [Multi-stage builds con Docker](#).

Este sería el programa en `Go`:

```
package main
import "fmt"
func main() {
```

```
fmt.Println("Hello world from Go!")
}
```

Y esta la versión inicial del `Dockerfile` (sin *multistage*):

```
FROM golang:1.14.2-alpine
WORKDIR /src
COPY src .
RUN go build -o /out/helloworld .
ENTRYPOINT ["/out/helloworld"]
```

Si construye la imagen `helloworld:inicial`:

```
$ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
$ docker build -t helloworld:inicial .
Sending build context to Docker daemon 5.632kB
Step 1/4 : FROM golang:1.14.2-alpine
...
Successfully built c640743c9c29
Successfully tagged helloworld:inicial
$ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
helloworld inicial c640743c9c29 3 seconds ago 372MB
golang 1.14.2-alpine dda4232b2bd5 12 months ago 370MB
$ docker run helloworld:inicial
Hello world from Go!
$
```

puede observar que la imagen `helloworld:inicial` ocupa 372M, 2M más de la imagen usada para construirla `golang`.

En la siguiente versión del `Dockerfile` se compila la aplicación en una imagen intermedia etiquetada como `builder` (`AS builder`) y a continuación se copia el resultado de la compilación desde la imagen intermedia (`COPY --from=builder`) en una imagen `alpine` que ocupa mucho menos:

```
FROM golang:1.14.2-alpine AS builder
WORKDIR /src
COPY src .
RUN go build -o /out/helloworld .

FROM alpine:3.12 AS bin
COPY --from=builder /out/helloworld /
CMD ["/helloworld"]
```

Si a partir de este nuevo `Dockerfile` se construye una imagen con etiqueta `helloworld:final`:

```
$ docker build -t helloworld:final .
Sending build context to Docker daemon 5.632kB
Step 1/7 : FROM golang:1.14.2-alpine AS builder
...
Step 6/7 : COPY --from=builder /out/helloworld /
--> 33bca22b3635
Step 7/7 : ENTRYPOINT ["/helloworld"]
--> Running in 450fea70f761
Removing intermediate container 450fea70f761
--> 52eef99e40f2
Successfully built 52eef99e40f2
Successfully tagged helloworld:final
$ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
helloworld final 52eef99e40f2 15 seconds ago 2.07MB
helloworld inicial c640743c9c29 6 minutes ago 372MB
golang 1.14.2-alpine dda4232b2bd5 12 months ago 370MB
$
```

Se puede comprobar que la imagen `helloworld:final` apenas ocupa 2M, y además de la reducción de tamaño **tiene una menor superficie de ataque**.

8.5.4 5.4. Cuidado con el uso de la etiqueta :latest

Existe cierta confusión sobre el uso de la etiqueta `latest`: no se debe suponer que corresponde a la última versión de una imagen. De hecho se asocia al último *build* de una imagen **en el que no se especifique una etiqueta**. Tiene una explicación detallada en el enlace [The misunderstood Docker tag: latest](#).

Por lo tanto, la **recomendación es indicar siempre una imagen específica**, por ejemplo `FROM ubuntu:20.04`

8.5.5 5.5. Combinar comandos RUN

Es recomendable reducir las capas de `docker` combinando comandos `RUN`:

```
RUN apt-get update
RUN apt-get install -y nginx
```

Mejor:

```
RUN apt-get update && \
    apt-get install -y nginx
```

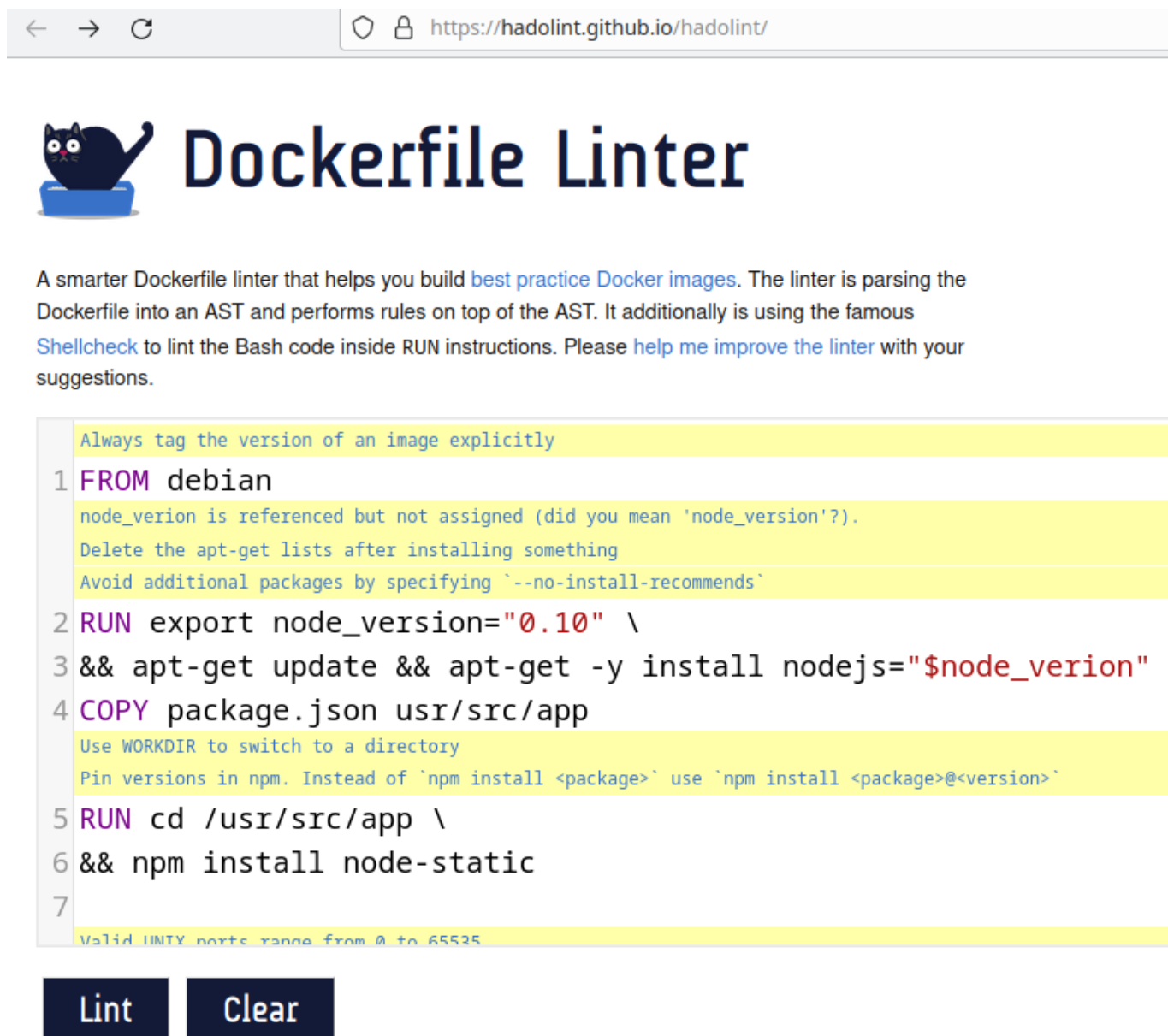
8.5.6 5.6. Linting

Existen herramientas para detectar fallos o posibles mejoras en un fichero `Dockerfile`. Una de ellas es **hadolint**: <https://github.com/hadolint/hadolint>

Puede instalarla y ejecutarla en su equipo, probarla online o usar un contenedor que incluya `hadolint` para verificar sus ficheros. Observe esta última opción para comprobar el fichero `dockerfile-test-hadolint`:

```
$ docker run --rm -i hadolint/hadolint < dockerfile-test-hadolint
-:1 DL3006 warning: Always tag the version of an image explicitly
-:2 DL3015 info: Avoid additional packages by specifying '--no-install-recommends'
-:2 DL3009 info: Delete the apt-get lists after installing something
-:2 SC2154 warning: node_verion is referenced but not assigned (did you mean 'node_version'?).
-:4 DL3045 warning: 'COPY' to a relative destination without 'WORKDIR' set.
-:5 DL3003 warning: Use WORKDIR to switch to a directory
-:5 DL3016 warning: Pin versions in npm. Instead of 'npm install <package>' use 'npm install <package>@<version>'
-:8 DL3011 error: Valid UNIX ports range from 0 to 65535
$
```

También puede probarla online: <https://hadolint.github.io/hadolint/>. En la figura puede observar el resultado de hacer un lint a un fichero `Dockerfile` en la web. Cada uno de los mensajes del linter (en amarillo) es un enlace a una descripción mas detallada:



The screenshot shows the Dockerfile Linter web application. At the top, there's a browser address bar with the URL `https://hadolint.github.io/hadolint/`. Below the address bar is the Dockerfile Linter logo, which features a black cat sitting on a blue box. The main heading is "Dockerfile Linter".

Below the heading, there's a paragraph of text: "A smarter Dockerfile linter that helps you build [best practice Docker images](#). The linter is parsing the Dockerfile into an AST and performs rules on top of the AST. It additionally is using the famous [Shellcheck](#) to lint the Bash code inside RUN instructions. Please [help me improve the linter](#) with your suggestions."

Below the text, there's a code editor area. It contains a Dockerfile snippet with line numbers 1 through 7. The code is as follows:

```
1 FROM debian
   node_version is referenced but not assigned (did you mean 'node_version'?).
   Delete the apt-get lists after installing something
   Avoid additional packages by specifying '--no-install-recommends'
2 RUN export node_version="0.10" \
3 && apt-get update && apt-get -y install nodejs="$node_version"
4 COPY package.json usr/src/app
   Use WORKDIR to switch to a directory
   Pin versions in npm. Instead of `npm install <package>` use `npm install <package>@<version>`
5 RUN cd /usr/src/app \
6 && npm install node-static
7
```

Below the code editor, there are two buttons: "Lint" and "Clear".

8.6 6. Escaneo de seguridad de las imágenes

Se deben comprobar las posibles vulnerabilidades de una imagen al construirla, **y también al ejecutarla**: entre ambos momentos pueden aparecer nuevas vulnerabilidades.

DockerHub incluye un servicio de escaneos de imágenes (`docker scout`) pero es una opción de pago. Usa los servicios de Snyk. Hay otras posibilidades: Anchore, Aqua Security (Trivy), JFrog Xray, etc. Puede encontrar más información en este enlace: [Container Security Scanners to find Vulnerabilities](#)

En este caso se prueba Trivy <https://github.com/aquasecurity/trivy> de la empresa Aqua Security. Puede detectar vulnerabilidades en los paquetes del Sistema Operativo o en las dependencias de las aplicaciones. Puede escanear contenedores, pero también repositorios git o sistemas de ficheros. Lo podemos instalar en nuestro sistema operativo, o ejecutarlo desde un contenedor Docker. Muestra las vulnerabilidades encontradas, dónde esta localizada, su CVE, versión del paquete que la corrige y enlaces a más información sobre la vulnerabilidad.

Para instalarlo debe acceder a las *releases* de trivy en <https://github.com/aquasecurity/trivy/releases> y seleccionar la que corresponda a su sistema (en debian la más actual en este momento es `trivy_0.60.0_Linux-64bit.deb`).

Al ejecutarse comprueba si tiene que actualizar su base de datos interna y luego comienza la exploración. Vea un ejemplo analizando la imagen `alpine`:

```
usuario@d12-base-01:~/Descargas$ trivy -q i alpine
```

Report Summary

Target	Type	Vulnerabilities	Secrets
alpine (alpine 3.21.3)	alpine	0	-

Legend:

- '-': Not scanned
- '0': Clean (no security findings detected)

```
usuario@d12-base-01:~/Descargas$
```

En cambio con otras imágenes con muchas vulnerabilidades la salida se hace complicada de gestionar y es mejor redirigir a un fichero:

```
usuario@d12-base-01:~/Descargas$ trivy -q i debian -o rest.txt
```

```
usuario@d12-base-01:~/Descargas$ more rest.txt
```

Report Summary

Target	Type	Vulnerabilities	Secrets
debian (debian 12.9)	debian	76	-

Legend:

- '-': Not scanned
- '0': Clean (no security findings detected)

debian (debian 12.9)

=====
Total: 76 (UNKNOWN: 0, LOW: 57, MEDIUM: 17, HIGH: 1, CRITICAL: 1)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version
Title					
apt versions, do not correctly...	CVE-2011-3374	LOW	affected	2.6.1	It was found that apt-key in apt, all
cve-2011-3374					https://avd.aquasec.com/nvd/
bash user than root]	TEMP-0841856-B18BAF			5.2.15-2+b7	[Privilege escalation possible to other
...					

8.6.1 6.1. Docker scan => scout

En el tutorial de docker se mostraba la herramienta `docker scan` que ha sido sustituida por Docker Scout

Docker Scout es una herramienta de análisis de seguridad que te permite examinar tus imágenes de contenedor en busca de vulnerabilidades (CVEs). A diferencia de los escaneos tradicionales, Scout ofrece recomendaciones en tiempo real sobre cómo corregir esas vulnerabilidades, sugiriendo, por ejemplo, versiones más seguras de la imagen base.

- **Vulnerabilidades (CVE):** El sistema identifica "Common Vulnerabilities and Exposures". Scout las clasifica en *Critical*, *High*, *Medium* y *Low*.
- **Análisis de capas:** Scout analiza cada capa de tu `Dockerfile`. Si una vulnerabilidad viene de la imagen base (ej. `node:14`), Scout te dirá: "Si actualizas a `node:20-bookworm`, eliminarás 50 vulnerabilidades".

Para usar Docker Scout:

1. Instalación del plugin: Docker Scout funciona como una extensión de la CLI de Docker.
2. Autenticación: Iniciar sesión en Docker Hub.
3. Análisis de imagen: Utilizar comandos para ver un resumen rápido y un desglose detallado de vulnerabilidades.
4. Interpretación de recomendaciones: Ver cómo Docker Scout nos sugiere mejorar el Dockerfile.

6.1.1. Instalar

<https://docs.docker.com/scout/install/>

Descargar e instalar el plugin de Docker Scout

```
curl -fsSL https://raw.githubusercontent.com/docker/scout-cli/main/install.sh -o install-scout.sh
sh install-scout.sh
```

6.1.2. Iniciar sesión

Docker Scout necesita consultar una base de datos de vulnerabilidades actualizada. Para ello, debes estar autenticado:

```
docker login
```

6.1.3. Comandos principales de análisis

Una vez instalado, aquí tienes los comandos fundamentales :

Comando	Propósito
<code>docker scout quickview [IMAGEN]</code>	Da un resumen rápido de las vulnerabilidades y la calidad de la imagen base.
<code>docker scout cves [IMAGEN]</code>	Muestra un listado detallado de todas las vulnerabilidades encontradas.
<code>docker scout recommendations [IMAGEN]</code>	Sugiere cambios en la imagen base para reducir el número de vulnerabilidades.

6.1.4. Ejemplo práctico

Suponga que tiene una imagen llamada `mi-app-debian:latest`. Para ver qué problemas tiene, ejecutar:

```
# Ver el estado general de seguridad
$ docker scout quickview mi-app-debian:latest

# Ver detalles específicos de cada fallo de seguridad
$ docker scout cves mi-app-debian:latest
```

8.7.7. Hardened images

<https://docs.docker.com/dhi/>

El servicio de **Docker Hardened Images (DHI)** es una iniciativa de Docker Inc. para proporcionar imágenes de contenedor extremadamente seguras, optimizadas y listas para producción.

El objetivo principal de estas imágenes es reducir la **superficie de ataque** de las aplicaciones (hasta en un 95% según Docker). Estas son sus características clave:

- **Casi cero vulnerabilidades (CVEs):** Están diseñadas para tener el mínimo posible de vulnerabilidades conocidas. Docker las parchea continuamente de forma automatizada.
- **Minimalismo extremo:** Se basan en un enfoque "distroless" o muy reducido (usando bases como Alpine o Debian), eliminando herramientas innecesarias como gestores de paquetes o shells que un atacante podría explotar.
- **Transparencia Total (SBOM):** Cada imagen incluye un **Software Bill of Materials (SBOM)** detallado, que es como una lista de ingredientes que permite saber exactamente qué librerías y versiones contiene la imagen.
- **Cumplimiento y Procedencia:** Cuentan con niveles de **SLSA (Supply-chain Levels for Software Artifacts) Nivel 3**, lo que garantiza que la imagen no ha sido alterada desde su creación hasta que llega a ti.
- **Sustitución directa (Drop-in):** Están diseñadas para reemplazar las imágenes base estándar (como `python`, `node`, `go`) sin necesidad de cambiar drásticamente tu `Dockerfile`.

Docker ha hecho que el catálogo principal de **Hardened Images sea gratuito y de código abierto (Apache 2.0)** para todos los desarrolladores. Puede usarla en proyectos personales o profesionales sin pagar una suscripción, siempre que no necesite características empresariales avanzadas.

Un ejemplo de cómo cambiaría el código sería:

Versión original

```
# Antes (Imagen estándar)
FROM node:20-alpine
WORKDIR /app
COPY . .
CMD ["node", "index.js"]
```

Versión "DHI"

```
# Usando la versión endurecida y verificada
FROM dhi.io/docker/node:20
WORKDIR /app
COPY . .
CMD ["node", "index.js"]
```

Para empezar a usarlas generalmente debes autenticarte en el registro específico de Docker para estas imágenes (dhi.io).

<https://docs.docker.com/dhi/get-started/>

Existen versiones comerciales para organizaciones con necesidades críticas:

- Ofrecen SLAs de parcheo (parches para vulnerabilidades críticas en menos de 7 días o incluso 24 horas).
- Permiten añadir certificados internos o paquetes específicos manteniendo la certificación de "hardened".
- Acceso a variantes con certificaciones **FIPS** (estándar federal de procesamiento de información) o **STIG**, necesarias en entornos gubernamentales o financieros muy regulados.
- Soporte de seguridad para imágenes incluso después de que la versión original (upstream) haya llegado a su fin de vida.

8.8 8. Varios de ampliación

8.8.1 8.1. OWASP Docker Security Cheat Sheet

La verdad es que debería haber empezado aquí. Algunas se han comentado, otras se escapan un poco del alcance del curso.

Enlace e índice: [Docker Security Cheat Sheet](#)

```
Table of contents
Introduction
Rules
RULE #0 - Keep Host and Docker up to date
RULE #1 - Do not expose the Docker daemon socket (even to the containers)
RULE #2 - Set a user
RULE #3 - Limit capabilities (Grant only specific capabilities, needed by a container)
RULE #4 - Prevent in-container privilege escalation
RULE #5 - Be mindful of Inter-Container Connectivity
RULE #6 - Use Linux Security Module (seccomp, AppArmor, or SELinux)
RULE #7 - Limit resources (memory, CPU, file descriptors, processes, restarts)
RULE #8 - Set filesystem and volumes to read-only
RULE #9 - Integrate container scanning tools into your CI/CD pipeline
RULE #10 - Keep the Docker daemon logging level at info
Rule #11 - Run Docker in rootless mode
RULE #12 - Utilize Docker Secrets for Sensitive Data Management
RULE #13 - Enhance Supply Chain Security
Podman as an alternative to Docker
References and Further Reading
```

8.8.2 8.2. Ejemplo practico mejorar Dockerfile avanzado

Un tutorial con una aplicación Node.js dónde se va mejorando el fichero `Dockerfile` paso a paso, comentando los posibles fallos, mejoras ,etc. El repositorio con la aplicación está en [Best Practices writing a Dockerfile](#)

8.8.3 8.3. Curso en la plataforma: Asegurando contenedores (tema 8)

Este apartado está extraído del curso disponible en la plataforma:

Autor: **Juan Diego Pérez Jiménez** para la Consejería de Educación y Deporte Licencia: creative commons: attribution - non commercial - share alike 4.0

Y se organiza en estos apartados:

8.3.1. Seguridad en las imágenes

En este apartado incide en algunas cuestiones ya vistas, puede aprovechar la explicación:

- Usar imágenes oficiales o de usuarios verificados.
- Usar DCT, *Docker's Content Trust*.
- Usar imágenes mínimas: añade el uso del fichero `.dockerignore`
- Restricción de privilegios con usuario distinto de root
- Escaneo de imágenes en DockerHub (servicio de pago)

8.3.2. Seguridad en el proceso de build

- ADD vs COPY .
- CMD & ENTRYPOINT
- Principio de mínimo privilegio
- ejecutar primero los comandos que necesitan root
- crear el usuario y conmutar a él con USER

```
# Imágen que va a usar
FROM XXXXX
....
# Creación del usuario para arrancar el servicio
RUN addgroup -S usuario && adduser -S usuario -G usuario.
.....
# Lista de órdenes que de ROOT
.....
# Establece test que ejecutará siguientes órdenes
USER test
# Define ENTRYPOINT que se ejecutará como test.
ENTRYPOINT .....
```

8.3.3. Seguridad al arrancar los contenedores

- Limitar recursos: memoria, CPU, etc.
- Deshabilitar las capacidades: añade explicación sobre los flags `--cap-drop` y `--cap-add` .

8.3.4. Vídeo

En ese apartado se muestra un vídeo donde se hacen prácticas de DCT y firma de imágenes. Le mostramos a continuación algunos de los comandos utilizados

```
$ docker system info
$ docker trust inspect hadolint/hadolint
$ docker trust key generate helloworld
$ ls -l ~/.docker/trust/private/
total 4
-rw----- 1 user user 422 may  2 20:42 g271e939289fc366096c538c58ff5e513bfc11bbca5ea891f756d30a809834c8.key
$ ls -l
...
-rwxrwxrwx 1 user user 196 may  2 20:42 helloworld.pub
drwxrwxrwx 1 user user  0 abr 30 22:04 src
$ docker trust signer add --key helloworld1.pub usuariotest userdocker/helloworld
$ docker build -t userdocker/helloworld:2.0 .
$ docker trust sign userdocker/helloworld:2.0
$ docker pull --disable-content-trust ....
```

9. Amplia Compose

9.1 Encadenar arranque contenedores

9.1.1 1. Introducción

Por defecto Docker intenta arrancar todos los servicios al mismo tiempo. Si por ejemplo una aplicación web intenta conectarse a una base de datos que todavía no está lista, la aplicación fallará.

9.1.2 2. Primer paso: `depends_on`

La instrucción para controlar el orden es `depends_on`. Con ella, se define explícitamente que un servicio (por ejemplo, web) depende de otro (por ejemplo, db). En ese caso el comando `docker compose up` arrancará los servicios en orden de dependencia: primero db, luego web. Y `docker-compose stop` los detendrá en el orden inverso.

```
services:
  web:
    image: miapp
    depends_on:
      - db # <-- Arranca 'web' solo después de iniciar 'db'
  db:
    image: postgres
    ...
```

El problema es que hay que distinguir:

- "Container Started"
- "Service Ready"

Aquí es donde muchos desarrolladores tropiezan. `depends_on` (en su forma básica) solo espera a que el contenedor esté en ejecución (running). No espera a que la aplicación dentro del contenedor esté lista (ready).

El fallo se produce si:

- Docker arranca el contenedor de Postgres (db).
- Docker ve que el contenedor está "running".
- Inmediatamente Docker arranca web.
- Postgres todavía está inicializando sus archivos y no acepta conexiones.
- la aplicación intenta conectarse, falla y se cierra (crash).

9.1.3 3. Solución: Usar `healthcheck` con `depends_on`

Esta es la forma más limpia y moderna de hacerlo si usas `docker-compose.yml`. Le dices a Docker Compose cómo comprobar la salud del contenedor de la base de datos y haces que tu aplicación espere a que esa comprobación sea exitosa.

1. Se añade un `healthcheck` al servicio de PostgreSQL. Para ello se usa la utilidad `pg_isready` que viene incluida en la imagen oficial de Postgres.
2. Usar la condición `service_healthy` en `depends_on`. En lugar de un simple `depends_on: [db]`, usar una sintaxis más avanzada.

```
services:
  db:
    image: postgres:15
    ...
    # --- INICIO DE LA SOLUCIÓN PASO 1 ---
    healthcheck:
      # Este comando comprueba si Postgres está listo
      test: ["CMD-SHELL", "pg_isready -U $POSTGRES_USER -d $POSTGRES_DB"]
      interval: 5s # Comprueba cada 5 segundos
      timeout: 5s # Espera 5 segundos por una respuesta
      retries: 10 # Intenta 10 veces antes de marcar como 'unhealthy'
    # --- FIN DE LA SOLUCIÓN ---
  app:
    image: miapp
```

```

ports:
  - '8000:8000'
...
# --- INICIO DE LA SOLUCIÓN PASO 2 ---
depends_on:
  db:
    # Espera a que el healthcheck del servicio 'db' pase
    condition: service_healthy
    # El valor por defecto es service_started ( dependencia simple sin healthcheck)
# --- FIN DE LA SOLUCIÓN ---

```

9.1.4 4. Otra soluciones

Aunque la recomendada es la anterior, puede encontrar otras posibilidades.

4.1. Lógica de Reintento en la Aplicación

Esta solución implica que la aplicación sea responsable de manejar los fallos de conexión al arrancar. En lugar de intentar conectarse una sola vez y fallar si la BBDD no está lista, tu aplicación entra en un bucle:

1. Intenta conectar a PostgreSQL.
2. Si falla (ej. `ConnectionRefusedError`), espera 5 segundos.
3. Vuelve al paso 1.
4. Repite esto durante un tiempo (ej. 1 minuto) o un número de reintentos (ej. 12 intentos).
5. Si lo consigue, continúa con el arranque normal.
6. Si falla después de todos los reintentos, se cierra con un error.

4.2. Script "Wrapper" (La solución clásica, ya obsoleta.. o no)

Esta es una solución muy común que no requiere modificar el código de tu aplicación, sino solo la forma en que se inicia el contenedor.

Cómo funciona:

1. Añade un pequeño script a la imagen de la aplicación (ej. `wait-for.sh`).
2. Este script usa herramientas como `nc` (netcat) o `pg_isready` para comprobar si el puerto de la BBDD está abierto y aceptando conexiones.
3. El script se queda en un bucle hasta que la BBDD responde.
4. Una vez que la BBDD está lista, el script ejecuta el comando real de tu aplicación (ej. `npm start`, `python app.py`).

El script más famoso para esto es [wait-for-it.sh](#).

9.2 Complementos y ayudas

9.2.1 1. docker compose y variables de entorno `.env`

1.1. Usando variables del fichero `.env`

Puede utilizar variables de entorno en un fichero docker compose. Para ello debe tener en su proyecto un fichero de nombre `.env` con una lista de pares clave-valor similar a este ejemplo:

```
# fichero .env
PG_USERNAME="admin"
PG_PASSWORD="superadmin"
PG_DATABASE="shop"
```

Puede usar estas variables de entorno en su fichero docker compose usando la sintaxis `${VAR_de_ENTORNO}`:

```
# compose.yaml
services:
  db:
    image: postgres
    environment:
      POSTGRES_USER: ${PG_USERNAME}
      POSTGRES_PASSWORD: ${PG_PASSWORD}
      POSTGRES_DB: ${PG_DATABASE}
    volumes:
      - ./data:/var/lib/postgresql/data
    ...
```

1.2. Incluyendo todo el fichero: `env_file`

En el ejemplo anterior incluye las variables que necesite del fichero y se las asigna a las variables de entorno del servicio.

Otra posibilidad es pasar el fichero completo de variables a un servicio del docker compose:

Ejemplo:

```
# compose.yaml
services:
  ...
  db:
    image: mariadb
    restart: always
    env_file:
      - .env.db # Fichero con las variables de entorno que necesita el servicio
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
```

+info:

- <https://www.warp.dev/terminus/docker-compose-env-file>
- <https://docs.docker.com/compose/how-tos/environment-variables/variable-interpolation/>